

Autómatas de Estados Finitos

2

Es el autómata más simple. La teoría subyacente detrás de estas máquinas es bien conocida y entendida. Se las ha estudiado en diseño de computadoras, neurofisiología, comunicaciones, lingüística, y teoría de computación. La máquina de estados finitos surgió naturalmente del modelado de aquellos dispositivos físicos simples que procesan señales que denotan información.

La mayoría de los dispositivos complejos, sean computadoras, máquinas o sistemas de comunicación, son realizados mediante la interconexión de varios componentes simples. Pero, solamente un número finito de componentes físicos pueden ser encerrados en un volumen dado. Los *símbolos* empleados para indicar eventos entre componentes son a menudo representados por los valores de cantidades físicas tales como posiciones mecánicas, voltajes, corrientes, temperaturas, presiones, flujo de fluidos o composiciones químicas. Dado que ninguna señal puede ser medida con gran exactitud dentro de tolerancias arbitrariamente pequeñas, los diseñadores de dispositivos físicos se limitan a un número reducido de valores fácilmente distinguibles, tanto en el caso de las señales propiamente dichas como en el de las condiciones internas (estados) de los dispositivos. Resumiendo, podemos decir que la realidad física impone una ineludible *finitud* en los sistemas físicos.

Una señal o estado requiere de un intervalo pequeño (pero no nulo) para su medida. Luego, sólo un número finito de operaciones puede ser efectuado confiablemente en una cantidad de tiempo finita. Nuevamente, la realidad física impone sistemas *discretos*, no sólo en los símbolos que ellos pueden procesar, o en los estados que ellos pueden asumir, sino también en los tiempos en los cuales ellos pueden cambiar de estado.

Nuestra tendencia natural a descomponer la solución de un problema en una secuencia de etapas se manifiesta en nuestro modelo de máquina simple mediante *acciones secuenciales*.

Por cierto, al diseñar una máquina que opera efectuando una etapa por vez se emplea la forma más simple conocida de hacer que su comportamiento sea *determinístico*, es decir, no sujeto a incertidumbre.

Luego, el modelo de autómata de estados finitos tiene las siguientes características:



Fig. 1. Características del modelo de autómata

En virtud de estas características, el modelo de autómatas de estado finito puede ser (y ha sido) aplicado a los siguientes casos:

1. *Sistemas mecánicos*: (Por ej., máquinas sumadoras) en los cuales las señales entre los componentes son representadas por las posiciones de las partes móviles.
2. *Sistemas electrónicos digitales*: (Por ej., circuitos lógicos de computadoras) en los cuales las señales son representadas por voltajes "altos" o "bajos", o por la "presencia" o "ausencia" de corriente.

3. *Sistemas hidráulicos y neumáticos*: (Por ej., redes de cañerías y válvulas que son actuadas por deltas de presión) en los cuales las señales se representan por la presencia o ausencia de presión en una cañería.
4. *Sistemas químicos*: (Por ej., transmisión del código genético durante la reproducción celular) en los cuales las señales son representadas por la composición química de ciertos materiales.

Las máquinas de estado finito también son útiles para el modelado de sistemas físicos que presentan combinaciones de estos efectos. Por ej., redes de conmutación de circuitos (donde hay presente una acción electromecánica) o el sistema nervioso humano (acción electroquímica).

A pesar de sus fuertes raíces en el modelado de fenómenos físicos, la máquina de estado finito no se limita al modelado de sistemas físicos. Se aplica a procedimientos de procesamiento de información, tales como la codificación y decodificación de mensajes, o al análisis sintáctico de programas computacionales.

Que esta máquina sea finita implica que la máquina en sí misma lo sea, y que también lo sea cualquier medio de almacenamiento al cual tenga acceso. Vamos a encontrar una multitud de actividades de procesamiento de información que no se corresponden con este modelo de estado finito. A pesar de estas limitaciones, este modelo juega un rol fundamental en la teoría de computación como una medida absoluta de la complejidad de un proceso computacional.

Nuestra formulación matemática de máquina de estado finito comienza con la suposición de que existe un *número finito de partes* y un *número finito de señales permisibles*. Luego se podrá ver que el mismo modelo surgiría a partir de la suposición de que la máquina tiene una *memoria finita de su experiencia previa*.

Consideremos por ejemplo un controlador para una puerta automática en la entrada de un supermercado. Este tipo de controladores permite la apertura de la puerta cuando detecta que una persona se acerca por la parte delantera. Podrían considerarse entonces dos estados: *Abierto* y *Cerrado* que representan el estado de la puerta en un momento determinado. Asimismo, se detectan cuatro posibles condiciones de entrada: que una persona intente ingresar (lo que implica que estará frente a la puerta), que una persona intente salir (estará en la parte trasera), que no haya personas intentando ingresar o salir y que finalmente haya simultáneamente dos personas al frente y en la parte trasera de la puerta.

El controlador se movería de estado en estado en función de la situación actual y del evento que está ocurriendo. Cuando está en el estado *Cerrado* y recibe un evento indicando que una persona está en la parte trasera de la puerta, que no existan personas cercanas o que simultáneamente hay individuos por delante y por detrás de la puerta, permanece en el estado cerrado (para no lastimar a las personas que están detrás). En caso de que haya una persona intentando entrar, el controlador abre la puerta y pasa al estado *Abierto*. En este estado, si hay personas delante de la puerta, en la parte de atrás o ambas situaciones, la puerta debe permanecer abierta y solo se cerrará si no hay personas en las proximidades.

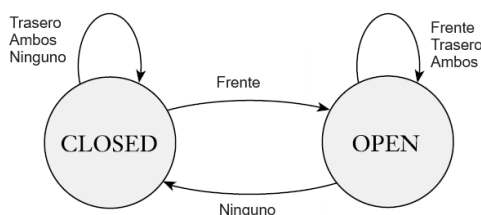


Fig. 2. Diagrama de cambio de estados de la puerta automática

Propiedades de los Autómatas de Estados Finitos

Consideremos una máquina M que es un traductor. Vamos a suponer que la máquina opera en los instantes $t_0, t_1, t_2, \dots, t_n$ donde t_0 (el origen) es arbitrario. Esos instantes de tiempo se indican, por simplicidad, mediante los enteros $t = 0, 1, 2, \dots$. En t_0 , las partes de esta máquina son inicializadas en una condición inicial conocida. En cada momento subsiguiente t , M recibe un símbolo de entrada s_t a través de su canal de entrada y transmite un símbolo de salida r_t a través de su canal de salida.

Los símbolos de entrada pertenecen a un alfabeto finito Σ , denominado *alfabeto de entrada*.

$$s_t \in \Sigma \text{ para todo } t$$

Igualmente, los símbolos de salida pertenecen al alfabeto finito Γ , o *alfabeto de salida*.

$$r_t \in \Gamma \text{ para todo } t$$

La secuencia de símbolos de entrada presentados a la máquina se denomina *estímulo*; mientras que la secuencia resultante de símbolos de salida se denomina *respuesta* de M al estímulo.

Dado que el comportamiento futuro de M depende claramente del estado presente de M , es natural preguntarnos acerca del estado de M antes de que ingrese alguna señal. Para que la máquina presente un comportamiento determinístico, es necesario que sus partes se encuentren en un estado fijo y conocido antes de que ningún estímulo se aplique. El estado total correspondiente a esta situación se denomina *estado inicial* de M y es indicado por q_0 .

Vamos a hacer la suposición adicional que la máquina hace cambios discretos por lo que su comportamiento puede ser especificado indicando la secuencia de estados:

$$q_0, q_1, q_2, \dots, q_t \quad \text{donde } q_i \in Q$$

que describe la condición interna de M en cada momento.

El canal de entrada ingresa a M y se conecta con ciertas partes de M . Cuando el símbolo de entrada s_{t+1} ingresa, influye en esas partes (que se encuentran en una condición representada por el estado q_t) y por lo tanto establece una nueva condición descrita por el estado q_{t+1} . El nuevo estado depende solamente de cual es el símbolo que arriba y de cual es el estado de M previo a su arribo.

$$q_{t+1} = f(q_t, s_{t+1}) \text{ para todo } t \geq 0$$

Esta función se denomina *función de transición de estados* de M . Su dominio es el conjunto de todos los pares estado-símbolo y su rango es un subconjunto de los estados.

$$f: Q \times \Sigma \rightarrow Q$$

Al canal de salida de esta máquina convergen señales desde ciertas partes de M . El símbolo de salida generado por el arribo de un símbolo de entrada depende solamente de qué símbolo arriba y de cual es el estado de M previo al arribo de la señal. Luego existe una función g que especifica el símbolo de salida producido por M en términos de su estado presente y del símbolo de entrada que ingresa.

$$r_{t+1} = g(q_t, s_{t+1}) \text{ para todo } t \geq 0$$

Esta función se denomina *función de salida* de M . Su dominio es el conjunto de todos los pares estado-símbolo de entrada, y su rango es el alfabeto de salida.

$$g: Q \times \Sigma \rightarrow \Gamma$$

Estas se formalizan en dos modelos conocidos: Máquina de Mealy (o con salida asociada a transición) y máquina de Moore (o con salida asociada a estados).

Máquinas con salida asociada a transición

Una máquina de estados finitos con salida asociada a transición es una tupla de seis elementos:

$$M_t = (Q, \Sigma, \Gamma, f, g, q_0)$$

donde

- Q Conjunto finito de estados internos
- Σ Alfabeto finito de entrada (eventos de entrada)
- Γ Alfabeto finito de salida
- $f: Q \times \Sigma \rightarrow Q$ es la función de transición de estados
- $g: Q \times \Sigma \rightarrow \Gamma$ es la función de salida
- $q_0 \in Q$ es el estado inicial

Este tipo de máquina, conocido en la literatura como *autómata de Mealy* se caracteriza por la asociación de símbolos de salida con transiciones entre estados.

Es importante recordar que estas máquinas de estados finitos incluyen la especificación de un estado inicial.

Para describir a una máquina de estados finitos particular, es necesario especificar la función de transición de estados y la función de salida. Existen dos representaciones estándar: la *tabla de estados* y el *diagrama de estados*.

La **tabla de estados** es una representación tabular de las dos funciones, empleando una fila para cada estado y una columna para cada símbolo de entrada. En la tabla, en la posición correspondiente al estado q y al símbolo s escribimos el nuevo estado $q' = f(q, s)$ y el símbolo de salida $r = g(q, s)$.

		s	
		...	
q	...	q', r	...
		...	

Fig. 3. Tabla de estados de la máquina M_t

El **diagrama de estados** es un grafo dirigido en el cual cada nodo corresponde a un estado de la máquina y cada arco indica una posible transición de un estado a otro. La siguiente figura muestra cómo las funciones de transición y de salida son representadas en un diagrama de estados:

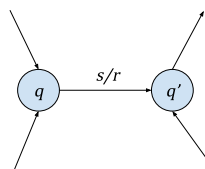


Fig. 4. Diagrama de estados de la máquina M_t

Cada arco está rotulado con el símbolo de entrada que causa la transición y el símbolo de salida que es generado. Escribimos:

$$q \xrightarrow{s/r} q' \quad \text{para indicar} \quad \begin{cases} f(q, s) = q' \\ g(q, s) = r \end{cases}$$

A menos que se especifique de otra manera, la primera fila de la tabla de estados es la correspondiente al estado inicial de la máquina. En el diagrama de estados este estado inicial se indica mediante un arco que no posee nodo de origen.

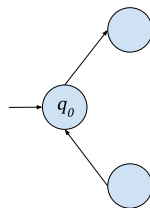


Fig. 5. Estado inicial de la máquina M_t

Cada camino dirigido a través del diagrama de estados muestra el comportamiento de la máquina en respuesta a la secuencia de símbolos de entrada que están rotulando el camino. Supongamos que $q_0, q_1, \dots, q_t$ son nodos sucesivos en un camino dirigido.

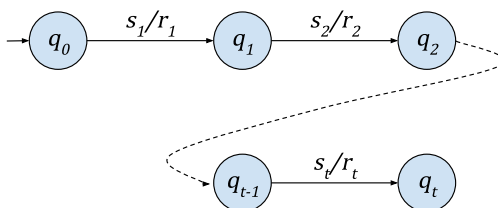


Fig. 6. Secuencia de estados de M_t

Luego, el estímulo $s_1 s_2 \dots s_t$ hace que la máquina describa una secuencia de estados $q_0, q_1, \dots, q_t$ son nodos sucesivos en un camino dirigido y genere la respuesta $r_1 r_2 \dots r_t$. Esto se representa mediante la notación:

$$q_0 \xrightarrow{s_1/r_1} q_1 \xrightarrow{s_2/r_2} \dots \xrightarrow{s_t/r_t} q_t$$

Es importante notar que los estados de una secuencia no necesariamente son distintos, dado que el camino trazado en el diagrama de estados puede contener lazos o loops.

Si indicamos al string de entrada como $\omega = s_1 s_2 \dots s_t$ y al string de salida como $\varphi = r_1 r_2 \dots r_t$, luego el comportamiento de la máquina se puede indicar en una notación más compacta:

$$q \xrightarrow{\omega/\varphi} q' \quad \text{donde} \quad \begin{cases} q = q_0 \\ q' = q_t \end{cases}$$

Ejemplo 1: Debemos diseñar una máquina de estados finitos M_1 , cuya salida sea el resultado de efectuar la operación $n \bmod m$ (para enteros $n \geq 0$ y $m > 0$, el valor de $n \bmod m$ es el del resto de la división n/m), donde $n =$ longitud de un string arbitrario ($n = |\omega|$) y $m = 3$.

$$\begin{aligned} \text{Sea} \quad \Sigma &= \{a\} \\ Q &= \{q_1, q_2, q_3\} \\ \Gamma &= \{0, 1, 2\} \end{aligned}$$

Sea t el momento en el cual el último símbolo de un estímulo ω se presenta a M_1 . Luego, el estado $q(t)$ se interpreta de la siguiente manera:

$$\begin{aligned} q(t) = q_0 &\text{ significa que } |\omega| \bmod 3 = 0 \\ q(t) = q_1 &\text{ significa que } |\omega| \bmod 3 = 1 \\ q(t) = q_2 &\text{ significa que } |\omega| \bmod 3 = 2 \end{aligned}$$

Esta máquina tiene la siguiente tabla y el siguiente diagrama de estados:

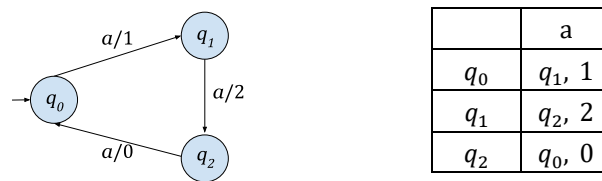


Fig. 7. Diagrama de estados y tabla de estados para la máquina M_1

Secuencia de entrada: a	$q_0 \xrightarrow{a/1} q_1$
Secuencia de entrada: aa	$q_0 \xrightarrow{a/1} q_1 \xrightarrow{a/2} q_2$
Secuencia de entrada: aaa	$q_0 \xrightarrow{a/1} q_1 \xrightarrow{a/2} q_2 \xrightarrow{a/0} q_0$
Secuencia de entrada: $aaaa$	$q_0 \xrightarrow{a/1} q_1 \xrightarrow{a/2} q_2 \xrightarrow{a/0} q_0 \xrightarrow{a/1} q_1$

Ejemplo 2: Sea ω una secuencia de símbolos en el alfabeto binario $\Sigma = \{0,1\}$. Si ω contiene un número par de unos, se dice que tiene paridad par; si ω contiene un número impar de unos, se dice que tiene paridad impar.

Deseamos diseñar una máquina de estados finitos M_2 que produzca una salida "1" cada vez que la paridad de la sentencia de entrada sea impar y que produzca una salida "0" cada vez que la paridad de la sentencia de entrada sea par. Luego, es obvio que $\Sigma = \Gamma = \{0, 1\}$.

Para diseñar esta máquina de estados finitos M_2 , observamos que la paridad de un string ωs , $s \in \{0,1\}$, queda determinada por el valor de s y por la paridad de ω .

Si $s = 0$, la paridad de ωs es la misma de ω

Si $s = 1$, la paridad de ωs es distinta de la de ω

Luego M_2 se define con sólo dos estados. Uno que indica que la secuencia de entrada hasta ese momento ha tenido paridad par y otro que indica que ha tenido paridad impar. La máquina M_2 tiene las siguientes características:



Fig. 8. Diagrama de estados y tabla de estados para la máquina M_2

Para la secuencia de entrada $\omega = 01100$ el comportamiento de M_2 está dado por:

$$q_0 \xrightarrow{0/0} q_0 \xrightarrow{1/1} q_1 \xrightarrow{1/0} q_0 \xrightarrow{0/0} q_0 \xrightarrow{0/0} q_0$$

y para la secuencia $\omega = 010011$:

$$q_0 \xrightarrow{0/0} q_0 \xrightarrow{1/1} q_1 \xrightarrow{0/1} q_1 \xrightarrow{0/1} q_1 \xrightarrow{1/0} q_0 \xrightarrow{1/1} q_1$$

Ejemplo 3: Los alfabetos de entrada y salida de la máquina M_3 son $\Sigma = \Gamma = \{0, 1\}$. Debemos diseñar esta máquina de forma tal que la secuencia de salida sea una réplica de la secuencia de entrada, pero demorada dos unidades.

$$r_t = s_{t-2} \quad \text{para } t > 2$$

No nos importan los valores de r_1 y r_2 dado que no se aplicaron símbolos de entrada antes del instante $t = 1$. Los estados de M_3 deben distinguir las secuencias de entrada de acuerdo con los dos últimos símbolos que fueron presentados s_{t-1} y s_{t-2} :

s_{t-2}	s_{t-1}	Estado de M_3
0	0	q_0
1	0	q_1
1	1	q_2
0	1	q_3

Un estímulo típico sería: 000110100 y su correspondiente respuesta: 000001101

$$q_0 \xrightarrow{0/0} q_0 \xrightarrow{0/0} q_0 \xrightarrow{0/0} q_0 \xrightarrow{1/0} q_3 \xrightarrow{1/0} q_2 \xrightarrow{0/1} q_1 \xrightarrow{1/1} q_3 \xrightarrow{0/0} q_1 \xrightarrow{0/1} q_0$$

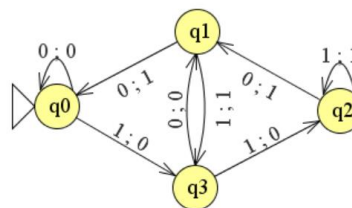


Fig. 9. Diagrama de estados para la máquina M_3

Nota: Una máquina que produce un retraso de n unidades requiere 2^n estados.

Máquinas con salida asociada a estado

Suele ser conveniente especificar la salida de una máquina de estados finitos en términos del estado de la máquina. Por ejemplo, en el caso de la máquina que verifica la paridad de una secuencia de 0's y 1's (máquina M_2), la salida de la máquina está únicamente determinada por el estado al que la transición conduce. En una máquina con salida asociada a estado la función de salida asigna un símbolo de salida con cada estado.

Definición: Una máquina de estados finitos con salida asociada a estado es una tupla de seis elementos:

$$M_s = (Q, \Sigma, \Gamma, f, h, q_0)$$

donde

- Q Conjunto finito de estados internos
- Σ Alfabeto finito de entrada
- Γ Alfabeto finito de salida
- $f: Q \times \Sigma \rightarrow Q$ es la función de transición de estados
- $h: Q \rightarrow \Gamma$ es la función de salida
- $q_0 \in Q$ es el estado inicial

En la literatura esta máquina se conoce como *autómata de Moore*. Estos autómatas se representan mediante tablas de estados y diagramas de estados de la siguiente forma:

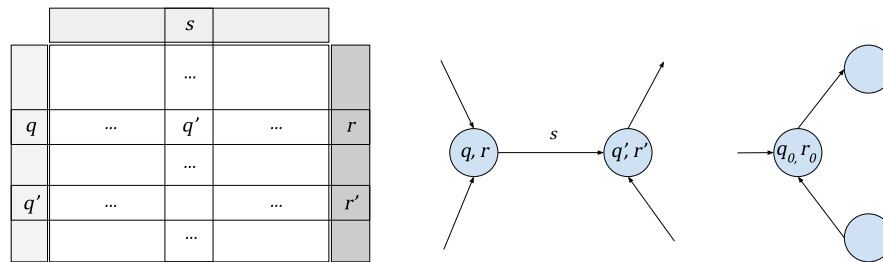


Fig. 10. Tabla de estados y Diagrama de estados para una máquina M_s

Las *secuencias de estados* se indican como pares (estado, símbolo de salida) unidos por flechas rotuladas con el símbolo de entrada que causa la transición indicada.

$$(q, r) \xrightarrow{s} (q', r') \text{ para indicar } \begin{cases} f(q, s) = q' \\ h(q) = r \\ h(q') = r' \end{cases}$$

Más adelante vamos a investigar si estos dos modelos (Autómatas de Mealy y de Moore) tienen una potencia computacional equivalente.

Ejemplo 4: Debemos diseñar una máquina M_4 que va a analizar secuencias de entrada definidas sobre el alfabeto binario $\Sigma = \{0, 1\}$. Si $\omega = s_1 s_2 \dots s_t$ es un string de entrada, y definimos:

$N_0(\omega)$ = Números de ceros en ω

$N_1(\omega)$ = Números de unos en ω

tales que $|\omega| = N_0(\omega) + N_1(\omega) = t$.

M_4 se diseñará de forma tal que la salida r_t de M_4 sea:

$$r_t = [N_1(\omega) - N_0(\omega)] \bmod 4$$

tal que $N_1(\omega) \geq N_0(\omega)$

Luego, el alfabeto de salida de M_4 es $\Gamma = \{0, 1, 2, 3\}$. Luego, los estados de esta máquina son:

$q(t) = A$ que significa $[N_1(\omega) - N_0(\omega)] \bmod 4 = 0$

$q(t) = B$ que significa $[N_1(\omega) - N_0(\omega)] \bmod 4 = 1$

$q(t) = C$ que significa $[N_1(\omega) - N_0(\omega)] \bmod 4 = 2$

$q(t) = D$ que significa $[N_1(\omega) - N_0(\omega)] \bmod 4 = 3$

Vamos a analizar la secuencia de estados correspondiente a un estímulo típico: 11011100

$$(A, 0) \xrightarrow{1} (B, 1) \xrightarrow{1} (C, 2) \xrightarrow{0} (B, 1) \xrightarrow{1} (C, 2) \xrightarrow{1} (D, 3) \xrightarrow{1} (A, 0) \xrightarrow{0} (D, 3) \xrightarrow{0} (C, 2)$$

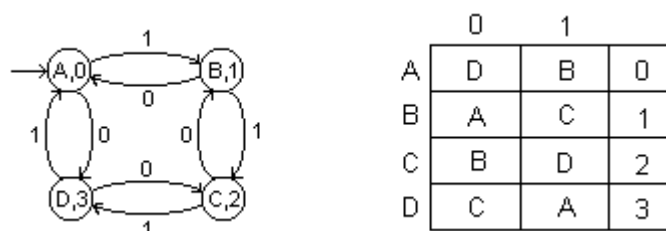


Fig. 11. Diagrama de estados y Tabla de estados para la máquina M_4

Ejemplo 5: Debemos diseñar una máquina M_5 tal que acepte un string binario sí y sólo si este comienza con 1 y contiene exactamente un sólo cero. El conjunto de todos esos strings es el conjunto $X = 11^*01^*$.

Esta máquina tendrá una salida 1, que indica aceptación, sí y sólo si el string de entrada pertenece al conjunto X y tendrá salida 0 en cualquier otro caso.

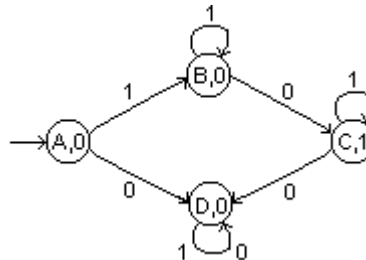


Fig. 12. Diagrama de estados para la máquina M_5

El estado D corresponde a un estado tal, que ninguna secuencia de símbolos de entrada puede lograr que M_5 deje el estado D. El estado B corresponde a secuencias de símbolos de entrada que solamente están formadas por 1's. El estado C corresponde a secuencias de símbolos pertenecientes a X .

La tabla de estados asociada con M_5 es:

	0	1	
A	D	B	0
B	C	B	0
C	D	C	1
D	D	D	0

Fig. 13. Tabla de estados para la máquina M_5

El estado D se conoce como estado trampa, y cualquier string de entrada que no comience con 1 o que posea más de un cero, conduce al estado D.

Cada string perteneciente al conjunto X conduce a M_5 a su estado de aceptación C, y son sólo esos strings los que llevan a M_5 a C.

Secuencias de estados

Hemos visto que la secuencia de estados asociados con un camino del diagrama de estados de una máquina corresponde a la aplicación de un string de símbolos de entrada y a la emisión de un string de símbolos de salida. Como próximo paso vamos a definir en forma más apropiada las nociones de estado sucesor y secuencias de estados. Estas definiciones luego serán necesarias para discutir la equivalencia entre máquinas y estudiar su relación con los lenguajes regulares. Estas definiciones son válidas para los dos tipos de autómatas: máquinas con salida asociada a transición y máquinas con salida asociada a estado.

Definición: Sea M una máquina de estados finitos con función de transición $f: Q \times \Sigma \rightarrow Q$. Si $f(q, s) = q'$, decimos que el estado q' es el *sucesor debido a s* del estado q , y escribimos:

$$q \xrightarrow{s} q'$$

Si el string de símbolos de entrada $\omega = s_1 s_2 \dots s_t$ hace que la máquina M evolucione desde el estado $q = q_0$ hasta el estado $q' = q_t$, es decir, si se cumple:

$$q_0 \xrightarrow{s_1} q_1 \xrightarrow{s_2} \dots \xrightarrow{s_t} q_t$$

decimos que q' es el *sucesor debido a ω* del estado q y lo indicamos como:

$$q \xRightarrow{\omega} q'$$

Bajo estas condiciones $q_0, q_1, \dots, q_t$ se denomina secuencia de estados admisible para ω .

Mediante estas definiciones hemos extendido el dominio de la función de transición de estados de forma que incluya todos los strings de entrada, en lugar de simples símbolos de entrada. Es decir, ahora tenemos:

$$f: Q \times \Sigma^* \rightarrow Q$$

donde:

$$f(q, \omega) = q' \Leftrightarrow q \xRightarrow{\omega} q'$$

Se verifica además que:

$$f(q, \lambda) = q; q \in Q$$

Luego, las siguientes cuatro afirmaciones son equivalentes:

1. Hay una secuencia de estados admisible desde q a q' para el string de entrada $\omega = s_1 s_2 \dots s_t$.
2. $q \xRightarrow{\omega} q'$
3. $f(q, \omega) = q'$
4. Hay un camino dirigido entre q y q' en el diagrama de estados de M cuyas transiciones están rotuladas mediante los símbolos de entrada $s_1, s_2, \dots, s_t$.

Definición: Sea M una máquina de estados finitos y supongamos que $q_0, q_1, \dots, q_t$, es una secuencia de estados admisible correspondiente a la secuencia de entrada $\omega = s_1 s_2 \dots s_t$; luego:

- a) Si M es una máquina con salida asociada a transición (Mealy), con función de salida $g: Q \times \Sigma \rightarrow \Gamma$, luego el string $\varphi = r_1 r_2 \dots r_t$ es conocido como respuesta de M al estímulo ω , donde:

$$r_i = g(q_{i-1}, s_i); \quad i = 1..t$$

- b) Si M es una máquina con salida asociada a estado (Moore), con función de salida $h: Q \rightarrow \Gamma$, luego el string $\varphi = r_1 r_2 \dots r_t$ es conocido como respuesta de M al estímulo ω , donde:

$$r_i = h(q_i); \quad i = 1..t$$

Conversión entre máquinas con salida asociada a transición y salida asociada a estado

La definición anterior señala una importante diferencia entre ambos tipos de máquinas: la máquina con salida asociada a estado tiene una respuesta para el string vacío. El propósito de los siguientes desarrollos es mostrar que, salvo por esta pequeña diferencia, ambas máquinas representan dos modelos de computación abstractos equivalentes.

Pero ¿cómo decidimos que los dos modelos son equivalentes? Surge naturalmente preguntarse si una máquina de un tipo puede siempre diseñarse de forma que imite perfectamente una máquina arbitraria del otro tipo.

Definición: Una máquina con salida asociada a transición M_t y una máquina con salida asociada a estado M_s son similares si para cada estímulo posible, la respuesta de M_s es exactamente igual a la de M_t pero precedida de un símbolo arbitrario, fijo.

El significado de esta definición está indicado en la siguiente figura:

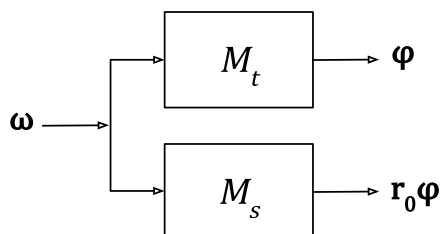


Fig. 14. Equivalencia entre máquinas M_t y M_s

El mismo string ω se aplica a ambas máquinas M_t y M_s . La respuesta de M_t es el string ϕ . La respuesta de M_s es el string $r_0\phi$ donde r_0 es un símbolo fijo determinado por M_s .

Teorema: Por cada máquina con salida asociada a estado M_s existe una máquina con salida asociada a transición M_t similar. En forma inversa, por cada máquina M_t existe una máquina M_s similar.

Probar cada una de estas afirmaciones consiste en construir una máquina de un tipo a partir de una dada máquina del otro tipo y demostrar que la máquina construida es similar a la original.

El problema más simple es obtener M_t a partir de M_s . Los estados y las transiciones de M_t son elegidos de forma tal que sean los mismos de M_s . Para construir M_t a partir de M_s se observará la siguiente regla: Si un estado q perteneciente a M_s tiene salida r , luego cada transición de M_t que finalice en el estado q se rotulará con la salida r .

Si partimos de: $M_s = (Q, \Sigma, \Gamma, f, h, q_0)$
 luego obtenemos: $M_t = (Q, \Sigma, \Gamma, f, g, q_0)$
 en la que se verifica que: $g(q, s) = h(f(q, s))$ para todo $q \in Q$ y $s \in \Sigma$

Dado que los estados y las transiciones en ambas máquinas M_s y M_t son idénticas, luego es evidente que una secuencia de estados es admisible en M_t para un estímulo ω sí y sólo sí es admisible en M_s para el estímulo ω . La similaridad de M_t y M_s se desprende naturalmente de este hecho y de la relación que vincula las funciones de salida.

Ejemplo 6: Dada la siguiente máquina M_s debemos construir una M_t equivalente:

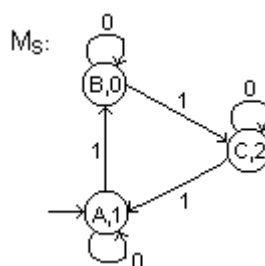


Fig. 15. Máquina M_s

Luego la máquina M_t es:

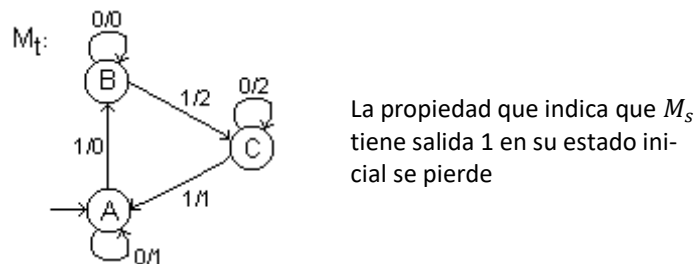


Fig. 16. Máquina M_t equivalente a M_s para el ejemplo 6.

El caso inverso, o sea la construcción de una máquina M_s a partir de una máquina M_t es algo más complicada, dado que no se puede seguir un camino inverso al presentado anteriormente. La razón es que M_t puede contener un estado q cuyas transiciones de entrada estén rotuladas por más de un símbolo de salida. Para evitar esta dificultad, permitimos que los estados de M_s sean el conjunto de pares estado-salida $Q_t \times \Gamma$ definidos en M_t . La máquina M_s va a ingresar al estado (q, r) cada vez que M_t ingresa al estado q y emite un símbolo de salida r .

La construcción se simboliza de la siguiente forma:

Partimos de: $M_t = (Q_t, \Sigma, \Gamma, f_t, g, q_0)$

y generamos: $M_s = (Q_s, \Sigma, \Gamma, f_s, h, (q_0, r_0))$

donde $Q_s = Q_t \times \Gamma$ y las funciones f_s y h están definidas como sigue:

Cada vez que se produce una transición:

$q \xrightarrow{s/r} q'$ en M_t , corresponde $((q, r'), r') \xrightarrow{s} ((q', r), r)$ en M_s para todo $r' \in \Gamma$.

$q_0 \xrightarrow{s/r} q'$ en M_t , corresponde $((q_0, r_0), r_0) \xrightarrow{s} ((q', r), r)$ en M_s para un r_0 elegido arbitrariamente.

Dado un string de entrada $\omega = s_1 s_2 \dots s_n$ y una secuencia de estados admisible $q_0, q_1, \dots, q_n$ correspondiente a ω en M_t , luego la secuencia de estados admisible correspondiente a ω en M_s consistirá en los pares:

$$(q_i, g(q_{i-1}, s_i)) \quad \text{para } i = 1..n$$

Por construcción, la respuesta de M_s será la misma que la de M_t con excepción del símbolo inicial r_0 .

Ejemplo 7: Se ilustra la construcción de una máquina con salida asociada a estado a partir de una máquina con salida asociada a transición. Se ha elegido arbitrariamente el símbolo 0 como la salida correspondiente al estado inicial.

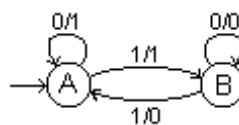


Fig. 17. Máquina M_t .

construimos M_s a partir de M_t :

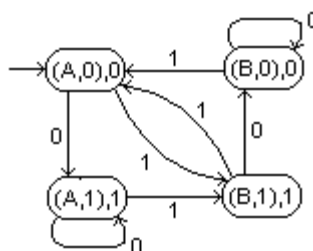


Fig. 18. Máquina M_s equivalente a M_t para el ejemplo 7.

Podemos concluir que los dos modelos de máquinas de estados finitos tienen una potencia computacional equivalente, la única excepción es que la máquina con salida asociada a estado brinda una respuesta específica para el string vacío. Dado que esta característica es importante en relación con el estudio de esta máquina como reconocedor de lenguajes, vamos a emplear este modelo en forma más extensiva.

Relación entre Autómatas Finitos y Lenguajes Regulares

Comenzaremos el estudio de la íntima relación que existe entre autómatas y lenguajes, a partir del nivel más simple. Este estudio no sólo producirá una mayor comprensión de estas máquinas y estos lenguajes, sino que también unificará conceptos vistos en los capítulos previos. veremos que las siguientes aseveraciones son equivalentes:

1. El lenguaje L es reconocido por algún aceptor de estados finitos.
2. El lenguaje L es generado por alguna gramática regular.
3. El lenguaje L es descrito por alguna expresión regular.

Aceptores de Estados Finitos

Vimos al comenzar el curso que una relación entre máquinas abstractas y lenguajes se puede establecer por medio de *aceptores*, máquinas que clasifican cada string de entrada como aceptado o rechazado.

Un autómata finito con salida asociada a estado y alfabeto de salida $\{0, 1\}$ puede ser interpretado como un aceptor de este tipo. Los estados de esta máquina pueden ser clasificados en dos clases:

1. Estados de aceptación: Cuya salida es 1.
2. Estados de rechazo: Cuya salida es 0.

Aceptores Determinísticos

El *lenguaje reconocido* por un aceptor de estados finitos consiste precisamente de los strings que toman a la máquina desde su estado inicial y la llevan a un estado de aceptación. En lugar de especificar los estados de aceptación por medio de una función de salida, será más simple identificar el subconjunto de los estados de la máquina que son estados de aceptación. En un diagrama de estados, los nodos correspondientes a estados de aceptación se dibujan con dobles circunferencias.

Definiremos entonces un Aceptor de estados finitos de la siguiente manera:

$$M_d = (Q, \Sigma, \delta, q_0, F)$$

donde:

- Q es el conjunto finito de estados
- Σ es el alfabeto finito de entrada
- $\delta = Q \times \Sigma \rightarrow Q$ es la función de transición de M
- $q_0 \in Q$ es el estado inicial
- $F \subseteq Q$ son los estados finales

Aceptores no Determinísticos

Nuestra primera etapa es relacionar aceptores de estados finitos con gramáticas regulares.

Hemos visto que una gramática consiste de un conjunto de reglas de producción, que pueden ser aplicadas en cualquier orden consistente, con el propósito de derivar un string terminal. La gramática es *permissiva* en el sentido que hay puntos en una derivación dada en los cuales debe seleccionarse la producción a ser aplicada. Por lo contrario, los autómatas finitos M_d estudiados hasta ahora son imperativos, en el sentido que cada transición está únicamente determinada por el estado predecesor y el símbolo de entrada que ingresa; es decir, no se permite comportamiento alternativo según lo especifica la función δ de nuestra definición.

Para simplificar el tratamiento de los dos sistemas (máquinas y gramáticas), vamos a introducir una generalización de aceptor de estados finitos que le otorga naturaleza permisiva. La generalización permite que un estado dado tenga un número arbitrario de sucesores para un símbolo de entrada.

Ejemplo 8:

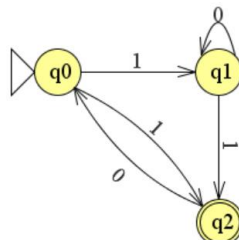


Fig. 19. Aceptor de estados finitos no determinístico M_{nd} .

Esta máquina es permisiva. Si está en el estado q_0 , dos transiciones son posibles para la entrada “1”:

$$q_0 \xrightarrow{1} q_1 \text{ y } q_0 \xrightarrow{1} q_2$$

También se observa que para algunas combinaciones de estados y símbolos de entrada *no* se especifica transición alguna. Específicamente, q_0 no tiene 0-sucesor y q_2 no tiene 1-sucesor.

Dado que el sucesor de un estado no es siempre único, las transiciones de este aceptor generalizado ya no pueden especificarse más por la función δ tal como la definimos en M_d . En cambio, debemos redefinir la función de transiciones de la máquina que contemple esta situación.

Definición: Un aceptor de estados finitos M_{nd} es una tupla de cinco elementos definida como:

$$M_{nd} = (Q, \Sigma, \delta, q_0, F)$$

donde

- Q es el conjunto finito de estados
- Σ es el alfabeto finito de entrada
- $\delta = Q \times \Sigma \rightarrow \wp(Q)$ es la función de transición de M
- $q_0 \in Q$ es el estado inicial
- $F \subseteq Q$ son los estados finales

Para cualquier conjunto Q , escribiremos $\wp(Q)$ como la colección de todos los subconjuntos de Q . En este caso, se toma $\wp(Q)$ como imagen de la función de transición debido a que una transición puede estar dirigida a cualquier subconjunto de estados de Q .

Dado que el comportamiento de un AEF no es necesariamente determinístico, a veces nos referimos a un AEF como un aceptor de estados finitos no-determinístico.

Las definiciones de s -sucesor, ω -sucesor y secuencia de estados admisibles ya presentadas se aplican sin inconvenientes a estos aceptores.

Si en la función de transición de un aceptor de estados finitos, cada estado tiene un único sucesor para cada símbolo de entrada. Luego, si existe una única elección de estado inicial, la máquina se comportará determinísticamente.

Al igual que en el caso de las máquinas de estados finitos ya vistas, un AEF determinístico asocia una única secuencia de estados con cada string de símbolos de entrada. Sin embargo, en un aceptor no determinístico, un estado puede tener más de un sucesor para cada símbolo de entrada y en consecuencia, no necesariamente habrá una secuencia de estados admisibles única, para cada string de entrada. En estas máquinas ya no podemos pensar en strings de entrada que causan transiciones de estados. En cambio, debemos pensar en strings de entrada que especifican caminos a través del diagrama de estados. Para algunos strings puede haber más de un camino, y para otros no.

En este tipo de máquinas, luego de leer un símbolo, la máquina se replica en múltiples copias de sí misma y sigue todas las posibilidades en paralelo. Cada copia de la máquina toma uno de los posibles caminos y se comporta como se describió previamente. El proceso se repite con cada símbolo de entrada. Si un estado no tiene un s -

sucesor, esa copia de la máquina muere. Finalmente, si cualquiera de las copias de la máquina llega a un estado de aceptación y finaliza el string de entrada, el aceptor de estados finitos no determinístico acepta el string de entrada.

Volvamos a la máquina del ejemplo de la Fig. 19, este aceptor tiene tres secuencias de estados admisibles para el string de entrada 101:

$$\begin{array}{ccccccc}
 & 1 & & 0 & & 1 & \\
 q_0 & \xrightarrow{1} & q_1 & \xrightarrow{0} & q_1 & \xrightarrow{1} & q_2 \\
 & 1 & & 0 & & 1 & \\
 q_0 & \xrightarrow{1} & q_2 & \xrightarrow{0} & q_0 & \xrightarrow{1} & q_1 \\
 & 1 & & 0 & & 1 & \\
 q_0 & \xrightarrow{1} & q_2 & \xrightarrow{0} & q_0 & \xrightarrow{1} & q_2
 \end{array}$$

Luego, tenemos los dos casos siguientes:

$$q_0 \xRightarrow{101} q_2 \quad q_0 \xRightarrow{101} q_1$$

Luego, para un dado AEF podemos tener estas posibilidades:

$$q_0 \xRightarrow{\omega} q' \text{ y } q_0 \xRightarrow{\omega} q'' \quad \text{donde } q' \neq q''$$

y ya no es apropiado decir que ω lleva a la máquina M desde el estado q al estado q' . Ahora debemos decir que ω puede llevar a M desde q a q' y puede llevar a M desde q a q'' .

Luego, las siguientes tres afirmaciones son equivalentes:

1. El aceptor M tiene una secuencia de estados admisibles desde el estado q al estado q' para el string de entrada ω .
2. El estado q' es un ω -sucesor (sucesor debido a ω) del estado q :

$$q \xRightarrow{\omega} q'$$
3. Hay un camino dirigido desde q hasta q' en el diagrama de estados de M cuyas transiciones están rotuladas por los símbolos que componen ω .

Luego, un AEF acepta un string ω si existe alguna secuencia de estados admisibles para ω desde algún estado inicial hasta algún estado final.

Lenguaje Reconocido por un AEF

Definición: Sea $M = (Q, \Sigma, \delta, q_0, F)$ un M_{nd} . Luego M acepta un string $\omega \in \Sigma^*$ sí y sólo sí $q_0 \xRightarrow{\omega} q'$ para algún $q' \in F$. Luego el lenguaje reconocido por M es el conjunto:

$$L(M) = \{ \omega \in \Sigma^* / q_0 \xRightarrow{\omega} q'; q' \in F \}$$

Dado que los estados finales de un aceptor son aquellos que indican aceptación, luego los términos estado final y estado de aceptación son equivalentes.

Consideremos el comportamiento del aceptor del ejemplo anterior ante el string 1011. Las secuencias de transiciones permitidas pueden representarse mediante el siguiente árbol:

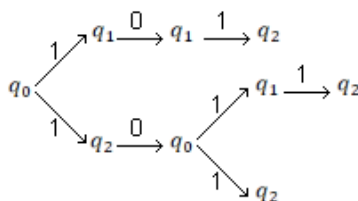


Fig. 20. Secuencia de transiciones para aceptar 1011 en M del ejemplo 1.

El árbol muestra que ciertas secuencias terminan antes del final del string 1011, pues el estado q_2 no tiene 1-sucesor. Dado que q_2 es el estado de aceptación, el diagrama muestra que los strings 1, 101 y 1011 son aceptados, y que el string 10 es rechazado.

Conversión a Aceptores Determinísticos

¿Cómo podemos relacionar el comportamiento de un autómata no determinístico con la operación de un dispositivo físico?

- 1) Podemos imaginar que en un dado instante de tiempo la máquina se encuentra en un dado estado y cuando, para un dado símbolo de entrada, se enfrenta a transiciones alternativas, elige arbitrariamente su próximo estado. Para determinar si un dado string es aceptado, debemos suponer que la máquina opera un número de veces suficiente para que cada ruta dentro del diagrama sea probada.
- 2) Otra posibilidad es que imaginemos que, cuando hay varias transiciones posibles, la máquina se divide en copias idénticas que en forma simultánea se ocupan de los caminos alternativos.
- 3) En el caso de autómatas finitos, hay un tercer punto de vista que también es útil. Podemos considerar que en cierto instante de tiempo la máquina se encuentra en cierta *combinación de estados* (esto en función de la incertidumbre sobre el verdadero estado en el que se encuentra la máquina). Esta idea conduce directamente a un procedimiento de obtención de una máquina determinística equivalente a cualquier AEF.

Volvamos a la máquina M , y analicemos su comportamiento frente al string 1011. Su comportamiento puede representarse como:

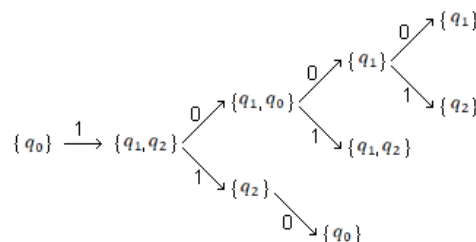


Fig. 21. Posibles caminos a través de M_{nd} .

Luego el comportamiento de M_{nd} frente a 1011 puede resumirse de la siguiente forma:

$$\{q_0\} \xrightarrow{1} \{q_1, q_2\} \xrightarrow{0} \{q_1, q_0\} \xrightarrow{1} \{q_1, q_2\} \xrightarrow{1} \{q_2\}$$

La aparición de un conjunto de estados en esta secuencia significa que en el diagrama de estados existen caminos por medio de los cuales, los símbolos de entrada del string correspondiente, conducen desde el estado q_0 a cada estado del conjunto. Por ejemplo, la aparición de $\{q_1, q_0\}$ en la secuencia significa que las dos opciones:

$$q_0 \xrightarrow{10} q_1 \text{ y } q_0 \xrightarrow{10} q_0$$

son posibles.

Luego, el subconjunto de estados al que conduce un string de entrada desde un estado inicial, se denomina *conjunto accesible* para ese string. Vamos a emplear el concepto de estados alcanzables para describir el comportamiento de M_{nd} para un string de entrada arbitrario.

Proseguimos con nuestro ejemplo. Inicialmente, M_{nd} debe estar en el estado q_0 . El 1-sucesor de q_0 es q_1 o q_2 , y no hay 0-sucesores para q_0 . Esto se resume en:

$$\{q_0\} \xrightarrow{1} \{q_1, q_2\}$$

Si M_{nd} está en q_1 o en q_2 , y una entrada 0 ocurre, el estado sucesor será q_0 o q_1 . Si en cambio ocurre una entrada 1, el estado sucesor puede ser solamente q_2 .

Si continuamos con este procedimiento, llegamos al árbol mostrado en la Fig. 21.

El conjunto de estados que puede ser *accedido* por medio de una p -transición a partir de otro conjunto de estados está solamente determinado por el diagrama de estados de la máquina. Por lo tanto, ocurrencias múltiples de un mismo conjunto de estados no necesitan ser distinguidas en el diagrama; y aquellos nodos idénticos del árbol anterior podrían ser fusionados en único nodo, como lo indica el siguiente diagrama:

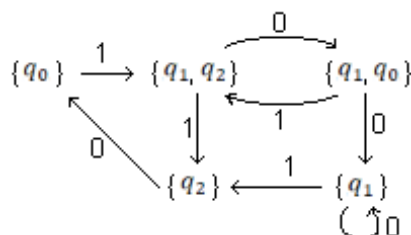


Fig. 22. Fusión de nodos idénticos en el diagrama de árbol.

Esta figura se asemeja al diagrama de estados de un aceptor determinístico, con la excepción que no se especifican transiciones para una entrada de 0 al conjunto accesible $\{q_0\}$ o para una entrada de 1 al conjunto accesible $\{q_2\}$. Estas transiciones no existen pues la máquina original M_{nd} no especifica ningún estado como 0-sucesor de q_0 o 1-sucesor de q_2 . Por lo tanto, aquellos strings que requieran el uso de estas transiciones faltantes no conducen a ningún estado del M_{nd} ; luego, el conjunto accesible o alcanzable para estos strings es el conjunto vacío. Luego, si incluimos a $\emptyset$ como un conjunto accesible, el diagrama anterior se convierte en el diagrama de estados de un aceptor de estados finitos determinístico. Luego los estados de la máquina determinística se encuentran en una correspondencia uno a uno con los conjuntos accesibles de M_{nd} .

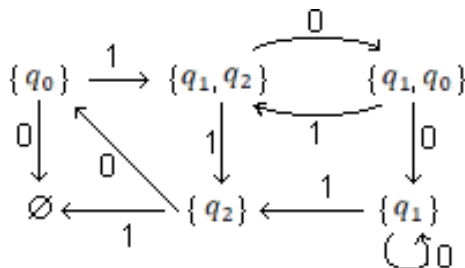


Fig. 23. Diagrama completo.

Luego, si $\emptyset$ es el conjunto accesible para algún string ω , luego $\emptyset$ es también el conjunto alcanzable para cualquier string que tenga a ω como prefijo. Luego, todas las transiciones desde $\emptyset$ retornan a $\emptyset$ y por lo tanto $\emptyset$ es un estado trampa del aceptor. Renombrando los estados obtenemos la M_d equivalente.

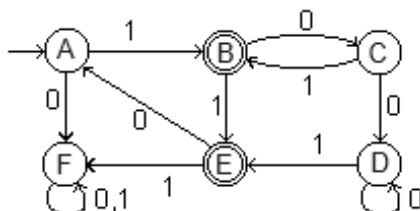


Fig. 24. Aceptor determinístico equivalente al M_{nd} .

El ejemplo anterior sugiere que uno siempre podría convertir un M_{nd} en un M_d que reconoce el mismo lenguaje. Luego, sea:

$$M_{nd} = (Q_{nd}, \Sigma, \delta_{nd}, q_{0_{nd}}, F_{nd})$$

un AEF cualquiera y supongamos que queremos crear un aceptor determinístico:

$$M_d = (Q_d, \Sigma, \delta_d, q_{0_d}, F_d)$$

tal que $L(M_{nd}) = L(M_d)$.

- Si le presentamos un string ω a M_{nd} , éste puede moverse a uno cualquiera de varios estados. Sea el conjunto de estados posibles:

$$X_{[\omega]} = \{q' \in Q_{nd} \mid q_{0_{nd}} \xrightarrow{\omega} q'\}$$

Luego el conjunto $X_{[\omega]}$ contiene todos los estados alcanzables de M_{nd} para el string de entrada ω .

- Si no se ha aplicado ningún símbolo a M_{nd} , la máquina deberá estar en su estado inicial.

$$X_{[\lambda]} = \{q_{0_{nd}}\}$$

- Por cada string φs , podemos expresar $X_{[\varphi s]}$ en términos de $X_{[\varphi]}$. Es decir, un estado q' será accesible por medio del string φs si y sólo si $q'' \xrightarrow{s} q'$ es la transición de M_{nd} para algún $q'' \in X_{[\varphi]}$. Es decir:

$$X_{[\varphi s]} = \{q' \in Q_{nd} \mid q'' \xrightarrow{s} q'; q'' \in X_{[\varphi]}\} \quad (@)$$

Luego $X_{[\varphi s]}$ está únicamente determinado por el símbolo de entrada s y el conjunto $X_{[\varphi]}$.

El número de conjuntos alcanzables $X_{[\omega]}$ distintos será siempre *finito* porque cada uno es un subconjunto del conjunto finito Q_{nd} . Luego, asignándole a M_d un estado por cada conjunto alcanzable, y transiciones con la ecuación (@), es posible obtener un aceptor determinístico M_d .

Dado que $X_{[\omega]}$ contendrá un estado final de M_{nd} si y sólo si $\omega \in L(M_{nd})$, los estados finales de M_d corresponden a los conjuntos accesibles que contienen los estados finales de M_{nd} . Luego, los detalles de la construcción son los siguientes:

1. Los elementos de Q_d son los subconjuntos de Q_{nd} que son alcanzables por medio de algún string de entrada.

$$Q_d = \{X_{[\omega]} \mid \omega \in \Sigma^*\}$$

2. El estado inicial de M_d es $X_{[\lambda]} = \{q_{0_{nd}}\}$.

3. Los estados de aceptación de M_d son los conjuntos accesibles que contienen estados de aceptación en M_{nd} .

$$F_d = \{X \in Q_d \mid X \cap F_{nd} \neq \emptyset\}$$

4. El estado X' es el s -sucesor del estado X en M_d , si y sólo si X' consiste precisamente de los s -sucesores en M_{nd} de los miembros de X . Por lo tanto $X \xrightarrow{s} X'$ en M_d si y solo si:

$$X' = \{q' \mid q \xrightarrow{s} q' \text{ en } M_{nd}; q \in X\}$$

Luego, estos elementos nos llevan a la formulación del siguiente teorema, que no será demostrado:

Teorema: Por cada aceptor de estados finitos M_{nd} , se puede construir un AEF determinístico M_d , tal que $L(M_{nd}) = L(M_d)$.

Aceptor de Estados Finitos con Transiciones λ (AEF- λ)

Podemos extender la definición de un AEF M_{nd} para que contemple transiciones de estados sin un símbolo de entrada específico. La función de transición toma un estado y un símbolo de entrada o el string vacío y producen un conjunto de estados sucesores posibles. En ese sentido podemos definir un AEF_λ de la siguiente manera:

Definición: Un aceptor de estados finitos con transiciones λ (AEF_λ) es un aceptor no determinístico:

$$M = (Q, \Sigma, \delta, q_0, F)$$

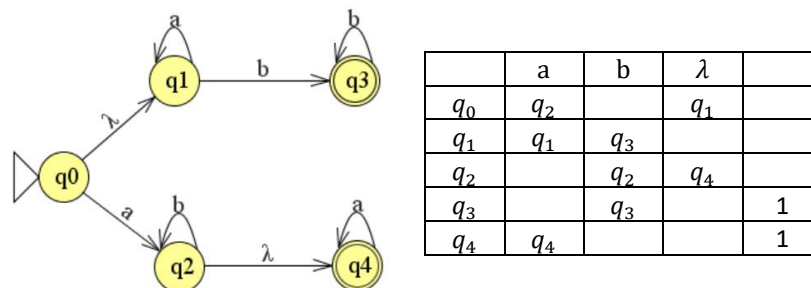
donde:

- Q es el conjunto finito de estados
- Σ es el alfabeto finito de entrada
- $\delta = Q \times \Sigma_\lambda \rightarrow \wp(Q)$ es la función de transición de M
- $q_0 \in Q$ es el estado inicial
- $F \subseteq Q$ son los estados finales

Para cualquier alfabeto Σ escribiremos Σ_λ como $\Sigma \cup \{\lambda\}$

La transición $\delta(q, \lambda) = \{q_1, q_2, \dots, q_n\}$, llamada transición λ o transición nula, tiene el siguiente significado: estando en el estado q , la máquina puede cambiar a cualquiera de los estados $q_1, q_2, \dots, q_n$ independientemente del símbolo leído. Dicho de otra manera, las transiciones λ permiten al autómata cambiar de estado sin leer ningún símbolo de la entrada.

En el diagrama de estados, las transiciones λ dan lugar a arcos rotulados con λ . Un string de entrada ω es aceptado por un AEF_λ si existe por lo menos un camino cuyos rótulos son exactamente los símbolos de ω , intercalados con cero, una o más λ s.



Analicemos cómo se comporta esta máquina ante el string de entrada $\omega = abb$:

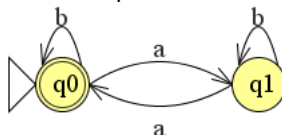
$$\begin{aligned}
 q_0 &\xrightarrow{\lambda} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_3 \xrightarrow{b} q_3 \\
 q_0 &\xrightarrow{a} q_2 \xrightarrow{b} q_2 \xrightarrow{b} q_2 \xrightarrow{\lambda} q_4 \\
 q_0 &\xRightarrow{abb} q_3 \quad \vee \quad q_0 \xRightarrow{abb} q_2
 \end{aligned}$$

Los AEF_λ permiten aún más libertad en el diseño de autómatas, especialmente cuando hay numerosas concatenaciones.

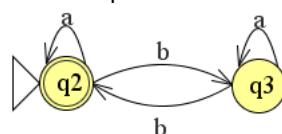
Ejemplo 9: Sea $\Sigma = \{a, b\}$ y $L = \{\omega \in \Sigma^* \mid \omega \text{ tiene un número par de a's o un número par de b's}\}$

Podemos construir por separado dos máquinas para que cada una de ellas acepte parte del lenguaje, luego, mediante el uso de transiciones λ podemos unir ambas partes y reconocer el lenguaje unión.

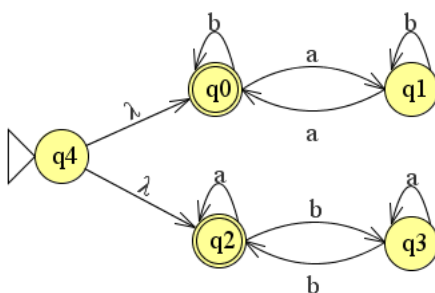
AEF determinístico que acepta strings con un número par de a's:



AEF determinístico que acepta strings con un número par de b's:



AEF_λ que acepta strings con un número par de a's o un número par de b's:



Equivalencia entre AEF_λ y AEF no Determinísticos

A continuación, se mostrará que el modelo AEF_λ es computacionalmente equivalente al modelo AEF no determinístico. O dicho más gráficamente, las transiciones λ se pueden eliminar, añadiendo transiciones que las simulen, sin alterar el lenguaje reconocido.

En primer lugar, un AEF no determinístico $M = (Q, \Sigma, \delta, q_0, F)$ puede ser considerado como un AEF_λ en el que, simplemente, hay cero transiciones λ . Para la afirmación recíproca tenemos el siguiente teorema:

Teorema 2: Dado un $AEF_\lambda M = (Q, \Sigma, \delta, q_0, F)$, se puede construir un AEF no determinístico M' equivalente a M , es decir, tal que $L(M) = L(M')$.

Para construir M' a partir de M se requiere la noción de λ -clausura de un estado.

Definición: Para un estado $q \in Q$, la λ -clausura de q , simbolizada $\lambda[q]$, es el conjunto de estados de M a los que se puede llegar desde q por 0, 1 o más transiciones λ .

$$\lambda[q] = \{q\} \cup \{q' \in Q \mid q \xrightarrow{\lambda} \dots \xrightarrow{\lambda} q' \text{ en } M\}$$

o también:

$$\lambda[q] = \{q\} \cup \lambda[q'] \quad \forall q' = \delta(q, \lambda)$$

Por definición, $q' \in \lambda[q]$ y $\lambda[\emptyset] = \emptyset$.

Adicionalmente podemos definir a λ -clausura de un conjunto de estados $\{q_1, q_2, \dots, q_k\}$ como:

$$\lambda[\{q_1, q_2, \dots, q_k\}] = \lambda[q_1] \cup \lambda[q_2] \cup \dots \cup \lambda[q_k]$$

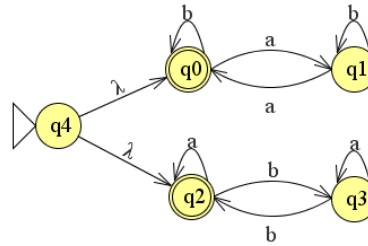
Procedimiento de construcción. Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AEF_λ , obtendremos $M' = (Q, \Sigma, \delta', q_0, F')$ donde:

$$\delta': Q \times \Sigma \rightarrow \wp(Q) \quad \text{tal que } \delta'(q, s) = \lambda[\delta(\lambda[q], s)] \quad q \in Q; s \in \Sigma$$

$$F' = \{q \in Q / \lambda[q] \cap F \neq \emptyset\}$$

M' simula así las transiciones λ de M teniendo en cuenta todos los posibles caminos. Por otro lado, los estados de aceptación de M' incluyen los estados de aceptación de M y aquellos estados desde los cuales se puede llegar a un estado de aceptación por medio de una o más transiciones λ .

Ejemplo 10: Vamos a aplicar el procedimiento de transformación a la máquina del ejemplo anterior:



Las λ -clausuras de los estados vienen dadas por:

$$\begin{aligned} \lambda[q_0] &= \{q_0\} \\ \lambda[q_1] &= \{q_1\} \\ \lambda[q_2] &= \{q_2\} \\ \lambda[q_3] &= \{q_3\} \\ \lambda[q_4] &= \{q_4, q_0, q_2\} \end{aligned}$$

La función de transición $\delta': Q \times \{a, b\} \rightarrow \wp(Q)$ es:

$$\begin{aligned} \delta'(q_0, a) &= \lambda[\delta(\lambda[q_0], a)] = \lambda[\delta(\{q_0\}, a)] = \lambda[\{q_1\}] = \{q_1\} \\ \delta'(q_0, b) &= \lambda[\delta(\lambda[q_0], b)] = \lambda[\delta(\{q_0\}, b)] = \lambda[\{q_0\}] = \{q_0\} \end{aligned}$$

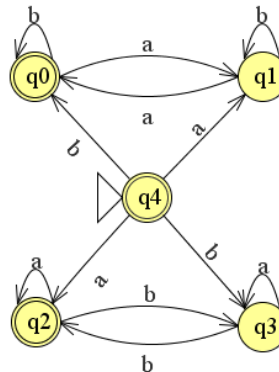
$$\begin{aligned} \delta'(q_1, a) &= \lambda[\delta(\lambda[q_1], a)] = \lambda[\delta(\{q_1\}, a)] = \lambda[\{q_0\}] = \{q_0\} \\ \delta'(q_1, b) &= \lambda[\delta(\lambda[q_1], b)] = \lambda[\delta(\{q_1\}, b)] = \lambda[\{q_1\}] = \{q_1\} \end{aligned}$$

$$\begin{aligned} \delta'(q_2, a) &= \lambda[\delta(\lambda[q_2], a)] = \lambda[\delta(\{q_2\}, a)] = \lambda[\{q_2\}] = \{q_2\} \\ \delta'(q_2, b) &= \lambda[\delta(\lambda[q_2], b)] = \lambda[\delta(\{q_2\}, b)] = \lambda[\{q_3\}] = \{q_3\} \end{aligned}$$

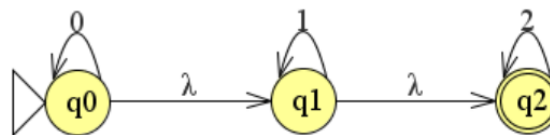
$$\begin{aligned} \delta'(q_3, a) &= \lambda[\delta(\lambda[q_3], a)] = \lambda[\delta(\{q_3\}, a)] = \lambda[\{q_3\}] = \{q_3\} \\ \delta'(q_3, b) &= \lambda[\delta(\lambda[q_3], b)] = \lambda[\delta(\{q_3\}, b)] = \lambda[\{q_2\}] = \{q_2\} \end{aligned}$$

$$\begin{aligned} \delta'(q_4, a) &= \lambda[\delta(\lambda[q_4], a)] = \lambda[\delta(\{q_4, q_0, q_2\}, a)] = \lambda[\{q_1, q_2\}] = \{q_1, q_2\} \\ \delta'(q_4, b) &= \lambda[\delta(\lambda[q_4], b)] = \lambda[\delta(\{q_4, q_0, q_2\}, b)] = \lambda[\{q_0, q_3\}] = \{q_0, q_3\} \end{aligned}$$

El autómata M' así obtenido es el siguiente donde el conjunto de estados de aceptación de M' incorpora a q_4 debido a que su λ -clausura no es disjunto a F . Finalmente $F' = \{q_0, q_2, q_4\}$



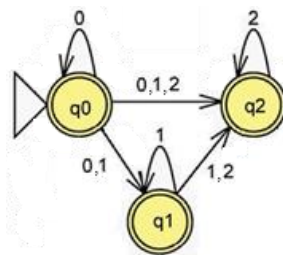
Ejemplo 11: Vamos a aplicar el procedimiento de transformación a la siguiente máquina:



$$\begin{aligned}\lambda[q_2] &= \{q_2\} \\ \lambda[q_1] &= \{q_1\} \cup \lambda[q_2] = \{q_1, q_2\} \\ \lambda[q_0] &= \{q_0\} \cup \lambda[q_1] = \{q_0, q_1, q_2\}\end{aligned}$$

$$\begin{aligned}\delta'(q_0, 0) &= \lambda[\delta(\lambda[q_0], 0)] = \lambda[\delta(\{q_0, q_1, q_2\}, 0)] = \lambda[\{q_0\}] = \{q_0, q_1, q_2\} \\ \delta'(q_0, 1) &= \lambda[\delta(\lambda[q_0], 1)] = \lambda[\delta(\{q_0, q_1, q_2\}, 1)] = \lambda[\{q_1\}] = \{q_1, q_2\} \\ \delta'(q_0, 2) &= \lambda[\delta(\lambda[q_0], 2)] = \lambda[\delta(\{q_0, q_1, q_2\}, 2)] = \lambda[\{q_2\}] = \{q_2\} \\ \delta'(q_1, 0) &= \lambda[\delta(\lambda[q_1], 0)] = \lambda[\delta(\{q_1, q_2\}, 0)] = \lambda[\emptyset] = \emptyset \\ \delta'(q_1, 1) &= \lambda[\delta(\lambda[q_1], 1)] = \lambda[\delta(\{q_1, q_2\}, 1)] = \lambda[\{q_1\}] = \{q_1, q_2\} \\ \delta'(q_1, 2) &= \lambda[\delta(\lambda[q_1], 2)] = \lambda[\delta(\{q_1, q_2\}, 2)] = \lambda[\{q_2\}] = \{q_2\} \\ \delta'(q_2, 0) &= \lambda[\delta(\lambda[q_2], 0)] = \lambda[\delta(\{q_2\}, 0)] = \lambda[\emptyset] = \emptyset \\ \delta'(q_2, 1) &= \lambda[\delta(\lambda[q_2], 1)] = \lambda[\delta(\{q_2\}, 1)] = \lambda[\emptyset] = \emptyset \\ \delta'(q_2, 2) &= \lambda[\delta(\lambda[q_2], 2)] = \lambda[\delta(\{q_2\}, 2)] = \lambda[\{q_2\}] = \{q_2\}\end{aligned}$$

Obtenemos la siguiente máquina con su tabla de estado correspondiente.
Finalmente $F' = \{q_0, q_1, q_2\}$



	0	1	2	
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$	1
q_1	$\emptyset$	$\{q_1, q_2\}$	$\{q_2\}$	1
q_2	$\emptyset$	$\emptyset$	$\{q_2\}$	1

Equivalencia de Máquinas de Estados Finitos

Un problema importante que aparece en todas las discusiones relacionadas con el estudio de autómatas es el de *equivalencia*: Bajo qué condiciones dos autómatas exhiben el mismo comportamiento.

	Tipo de Autómata	Condición
Si son	Reconocedores de Lenguajes	Sí y sólo sí reconocen el mismo lenguaje.
	Generadores de Lenguajes	Sí y sólo sí generan el mismo lenguaje.
	Traductores de Lenguajes	Sí y sólo sí producen idénticas traducciones de cada string de entrada.

No es siempre posible decidir cuándo dos autómatas presentan un comportamiento equivalente. Veremos que esta cuestión puede ser siempre resuelta cuando los autómatas sean máquinas de estados finitos. La razón es que cada uno de estos autómatas puede ser puesto en una forma estándar esencialmente única.

La solución del problema de equivalencia para máquinas de estados finitos también resuelve algunos problemas que surgen naturalmente en la solución de problemas de diseño. Por ejemplo:

1. Dado el diagrama de estados de una máquina, es posible detectar y eliminar estados redundantes sin alterar el comportamiento de la máquina.
2. Es posible encontrar, por eliminación de estados redundantes, un autómata finito mínimo (con número mínimo de estados) que sea equivalente al original.



A lo largo de los próximos desarrollos estas dos preguntas van a encontrar una respuesta afirmativa. Si bien el logro de una máquina con un número mínimo de estados no necesariamente conduce a la máquina física de construcción más simple, la misma constituye un buen punto de partida para el diseño de hardware.

Definición. Máquinas equivalentes: Dos máquinas M_1 y M_2 son equivalentes sí y sólo sí:

1. Sus alfabetos de entrada y de salida son los mismos:

$$\Sigma_1 = \Sigma_2; \quad \Gamma_1 = \Gamma_2$$

2. Ante cada estímulo, M_1 y M_2 producen respuestas idénticas.

Es decir, si: $s_1(t) = s_2(t) \quad t \geq 1$,

luego $r_1(t) = r_2(t) \quad t \geq 1$

porque en $t = 0$ las máquinas están inicializadas, pero no necesariamente el estado inicial es el mismo.

La equivalencia entre M_1 y M_2 es indicada mediante $M_1 \sim M_2$.

De acuerdo a esta definición, máquinas equivalentes que operan como se indica en Fig. 25, deben producir secuencias de salida idénticas, independientemente de cuál sea la secuencia de entrada presentada.

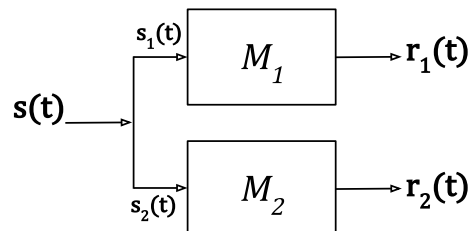


Fig. 25. Funcionamiento de máquinas M_1 y M_2 equivalentes.

De la definición de equivalencia se desprenden las propiedades de reflexividad, simetría y transitividad.

Reflexividad: $M \sim M$. Cada máquina es claramente equivalente a sí misma.

Simetría: Si $M_1 \sim M_2$, luego $M_2 \sim M_1$. El intercambio de subíndices en la definición no altera el significado de la equivalencia.

Transitividad: Si $M_1 \sim M_2$ y $M_2 \sim M_3$, luego $M_1 \sim M_3$. Cada vez que se cumpla que $r_1(t) = r_2(t)$ con $t \geq 1$ y $r_2(t) = r_3(t)$ con $t \geq 1$, luego se verifica que $r_1(t) = r_3(t)$ con $t \geq 1$.

La relación de equivalencia de máquinas particiona la clase de los autómatas de estados finitos en conjuntos de máquinas mutuamente equivalentes.

Particularmente trabajaremos sobre aceptores de estados finitos en nuestro desarrollo. En ese sentido, consideraremos que $\Gamma_1 = \Gamma_2 = \{0,1\}$

Estados Equivalentes

Para poder resolver el problema de equivalencia vamos a relacionar la equivalencia de los dos autómatas con sus estructuras internas expresadas en forma de diagramas o tablas de estados. Consideramos que dos estados de una máquina determinada son equivalentes si es imposible distinguir entre dos copias de la máquina, cada una de ellas inicializada en cada uno de esos estados. Un estado va a ser redundante si existe otro equivalente a él.

Definición: Dos estados q_a y q_b pertenecientes a un AEF $M = (Q, \Sigma, \delta, q_0, F)$ son *estados equivalentes* sí y sólo sí las máquinas:

$$M_a = (Q, \Sigma, \delta, q_a, F), \text{ y} \\ M_b = (Q, \Sigma, \delta, q_b, F)$$

son equivalentes; se indica como $q_a \sim q_b$.

La noción de equivalencia de estados se indica en la siguiente figura:

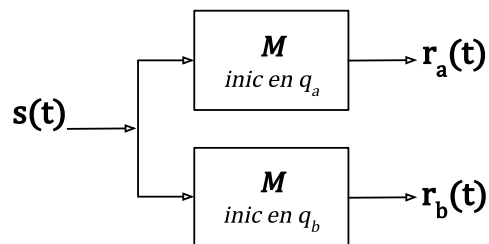


Fig. 26. Equivalencia de estados en la máquina M .

Test conceptual correspondiente a estados equivalentes.

Si en una máquina M , los estados q_a y q_b son equivalentes, luego, dos copias de M no podrán ser distinguidas por la presentación de distintos estímulos y la comparación de respuestas.

Dado que la equivalencia de estados es expresada en función de la equivalencia de máquinas, es fácil verificar que la relación de equivalencia de estados particiona los estados de una máquina en clases equivalentes que verifican:

1. Todos los estados pertenecientes a una misma clase son mutuamente equivalentes.
2. Dos estados cualesquiera pertenecientes a distintas clases **no** son equivalentes.

Si dos estados **no** son equivalentes, luego habrá una secuencia de entrada tal que cuando es aplicada a las dos copias de la máquina indicadas en la figura, producirán secuencias de salida diferentes.

Si dos estados de una máquina son equivalentes, luego uno de ellos es claramente redundante, dado que todas las transiciones que llevan a uno de los estados podrían ser derivadas al estado equivalente, sin afectar el comportamiento de la máquina. Para poder identificar estados redundantes en una máquina necesitamos un método que nos permita construir clases de estados equivalentes a partir del diagrama de estados o de la tabla de estados de la máquina.

Reducción de Estados y Verificación de Equivalencia

Los métodos que se discuten a continuación son aplicables a máquinas con salida asociada a estado y con salida asociada a transición; sin embargo, la presentación se efectúa en términos de AEF.

Máquinas Reducidas, Máquinas Conectadas

Intuitivamente, una máquina está en su forma más simple una vez que los estados redundantes y no usables son removidos.

Definición de máquina reducida: Una máquina de estados finitos es **reducida** si no contiene ningún par de estados equivalentes.

$$\text{M.R.: } \forall q, q' \in Q: q \not\approx q'$$

Definición de estado accesible: Un estado q en un autómata de estados finitos es **accesible** si hay algún string de entrada ω tal que:

$$q_0 \xRightarrow{\omega} q$$

Una máquina de estados finitos es **conectada** si todos sus estados son accesibles desde el estado inicial.

$$q \text{ es estado accesible si } \exists \omega \in \Sigma^* \text{ tal que } q_0 \xRightarrow{\omega} q$$

$$\text{M.C.: } \forall q \in Q: q \text{ es accesible}$$

Es evidente que ningún estado puede ser removido de una máquina reducida y conectada sin afectar el comportamiento de la misma. Los estados *inaccesibles* pueden ser siempre removidos, ya que la máquina nunca entra en ellos.

Secuencias de distinción y k -equivalencias

Si los estados q_a y q_b de una máquina M son equivalentes, ningún string de entrada aplicado al experimento indicado en la Fig. 26, puede producir respuestas diferentes en las dos copias de M .

En forma inversa, si los estados q_a y q_b no son equivalentes, habrá una secuencia de entrada, la cual aplicada al experimento de la Fig. 26 será rechazada y aceptada para las máquinas inicializadas en q_a y q_b respectivamente. En este caso q_a y q_b son llamados **estados distinguibles**.

Definición de estados k -distinguibles: Los estados q_a y q_b de un AEF $M = (Q, \Sigma, \delta, q_0, F)$ son k -distinguibles si existe un string $\omega \in \Sigma^*$ con $|\omega| \leq k$, que es aceptado por $M_a = (Q, \Sigma, \delta, q_a, F)$ y rechazado por $M_b = (Q, \Sigma, \delta, q_b, F)$.

Si los estados q_a y q_b no son k -distinguibles, decimos que son k -equivalentes.

Teorema: Dos estados de un autómata finito son k -equivalentes sí y sólo sí:

1. Son 1-equivalentes.
2. Para cada símbolo de entrada s , sus s -sucesores son $(k - 1)$ -equivalentes.

Claramente, dos estados de una máquina son **equivalentes** $\Leftrightarrow$ son k -equivalentes $\forall k \geq 1$.

Partición del Conjunto de Estados

El Teorema anterior sirve como base para los procedimientos de reducción de máquinas que desarrollaremos a continuación. Nuestro objetivo es particionar el conjunto de estados de una máquina en bloques de estados mutuamente equivalentes.

Supongamos que $M = (Q, \Sigma, \delta, q_0, F)$ es una máquina cuyo conjunto de estados Q va a ser particionado en bloques de estados equivalentes. Sea $Q = \{q_0, q_1, \dots, q_n\}$; dos estados q_i y q_j pertenecen al mismo bloque de la partición de Q siempre y cuando dos máquinas M_i y M_j inicializadas en q_i y q_j respectivamente, no produzcan respuestas distintas. Claramente los estados que sean de aceptación y rechazo formarán bloques distintos en una primera partición de estados P_0 .

Construiremos esta partición de Q mediante la sucesión de particiones $P_0, P_1, \dots, P_m, P_{m+1}$ tal que cada bloque de la partición k , $1 \leq k \leq m + 1$ contiene solo estados que son mutuamente k -equivalentes. Los estados q_i y q_j pertenecerán a un mismo bloque de P_k sí y sólo sí las máquinas M_i y M_j no pueden ser diferenciadas por ningún estímulo de k o menos símbolos.

Dado que los estados que son $(k - 1)$ -equivalentes son ciertamente k -equivalentes, cada bloque de P_{k+1} estará contenido en algún bloque de la partición P_k . De aquí que cada partición es un refinamiento¹ de su predecesor.

Con estas bases podemos construir el siguiente *algoritmo de particionado*.

¹ Si P_1 y P_2 son particiones del conjunto X y si cada bloque de P_2 es un subconjunto de exactamente un bloque de P_1 , luego P_2 es un refinamiento de P_1 . Es decir, si $P_1 = \{A_1, \dots, A_n\}$ y $P_2 = \{B_1, \dots, B_m\}$ son particiones del conjunto X y P_2 es un refinamiento de P_1 , luego para cada bloque B_j en P_2 hay un bloque A_i en P_1 tal que $B_j \subseteq A_i$.

El número de bloques en P_2 nunca es menor que el número en P_1 ; luego $m \geq n$. De acuerdo con esta definición, una partición es un refinamiento de sí misma.

Algoritmo de Particionado

- **Etapla 1:** Generar una partición inicial P_0 de Q agrupando aquellos estados de aceptación y de no aceptación. Los estados q y q' están en el mismo bloque de P_0 si son 0-equivalentes. Tomar un contador $k = 0$.
- **Etapla 2:** Obtener P_{k+1} a partir de P_k como sigue: Los estados q y q' están en el mismo bloque de P_{k+1} si y sólo si:
 1. Están en el mismo bloque de P_k .
 2. Para cada $s \in \Sigma$, los estados s -sucesores de q y q' están en el mismo bloque de P_k .
- **Etapla 3:** Repetir la etapa 2 hasta que $P_{m+1} = P_m$ para algún m , en otro caso pasar a la etapa 4.
- **Etapla 4:** Llamamos a P_m la partición final de Q y la renombraremos como P_f .

Cada partición P_k construida durante el procedimiento visto tiene bloques de estados que son mutuamente equivalentes. Además, por construcción, P_{k+1} es un refinamiento de P_k , $\forall k \geq 1$. Mostraremos que el procedimiento debe terminar con una partición P_m tal que $P_k = P_m$, $\forall k \geq m$.

1. Si $P_{k+1} = P_k$, luego $P_{k+j} = P_k$, $\forall j \geq 0$.
Si $P_{k+1} = P_k$, luego a partir de la etapa 2 del procedimiento anterior, $P_{k+2} = P_{k+1}$ y la aseveración sigue por inducción.
2. Si la máquina tiene n estados, luego $P_{n+1} = P_n$.
 $\forall k \geq 1$, P_{k+1} es un refinamiento de P_k . Luego, el número de bloques en P_{k+1} es mayor que el número de bloques en P_k , a menos que P_k sea final. Dado que el número de bloques en una partición no puede exceder el número de estados en M , y la partición P_n debe ser final, luego $P_{n+1} = P_n$.

Construcción de una Máquina Reducida

Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AEF. Vamos a construir una máquina reducida equivalente $M' = (Q', \Sigma, \delta', q_0', F')$. Los estados de M' van a representar las clases de equivalencia de estados de la máquina M , es decir, van a corresponder a los bloques de la partición final P_f que resulta de aplicar el procedimiento de particionado a M .

El estado inicial q_0' corresponde al bloque que contiene al estado inicial de M .

La tabla de estados de M' se obtiene de la siguiente manera: para encontrar el s -sucesor del estado q' de la máquina M' , seleccione cualquier estado en el bloque de la partición P_f que corresponde a q' y luego encuentre el bloque que contiene su s -sucesor. El estado correspondiente en M' es el s -sucesor de q' .

Los estados de aceptación de M' serán todas las particiones que resulten del refinamiento del bloque de estados de aceptación de la partición P_0 .

Ejemplo: consideremos el siguiente AEF M :

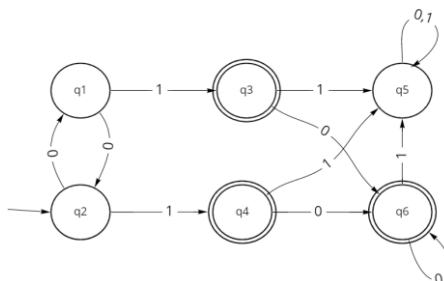


Fig. 27. Diagrama de estados de la máquina M con estados equivalentes.

Construiremos una primera partición P_0 , discriminando los estados de aceptación y de rechazo.

$$\begin{aligned}
 P_0 &= \{(q_1, q_2, q_5), (q_3, q_4, q_6)\} \\
 P_1 &= \{(q_1, q_2), (q_5), (q_3, q_4, q_6)\} \\
 P_2 &= \{(q_1, q_2), (q_5), (q_3, q_4, q_6)\} \\
 P_f &= P_2
 \end{aligned}$$

Claramente los estados q_1 y q_2 son equivalentes, así como los estados q_3 , q_4 y q_6 . Cada uno de los bloques de la última partición serán renombrados con los estados A, B y C respectivamente para construir el AEF equivalente sin estados redundantes.

Resulta entonces la siguiente tabla de estados y diagrama de estados:

	0	1
A	A	C
B	B	B
C	C	B

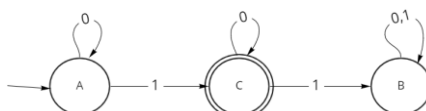


Fig. 28. Diagrama y tabla de estados de la máquina mínima M' .

Aceptores de Estados Finitos y Gramáticas Regulares

La relación entre aceptores de estados finitos y las gramáticas regulares se establece como una correspondencia entre conjuntos de strings indicados por las letras no terminales de una gramática y ciertos conjuntos de strings asociados con los estados de un aceptor.

Vamos a tratar explícitamente la relación entre gramáticas lineales por derecha y AEF. Un desarrollo similar podría efectuarse en términos de gramáticas lineales por izquierda.

Dada una gramática G , indicamos como $L(G, A)$ el conjunto de strings terminales derivables en la gramática G a partir de la letra no terminal A .

$$L(G, A) = \{ \omega \in T^* \mid A \xRightarrow{*} \omega \text{ en } G \}$$

Para cualquier gramática lineal por derecha, vamos a ver como estos conjuntos pueden ser identificados con ciertos conjuntos de strings, llamados conjuntos finales, que se asocian con los estados de un aceptor de estados finitos M .

Definición: Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AEF. El conjunto final $E(q)$ de un estado q perteneciente a M es la colección de strings de entrada que pueden llevar a M desde el estado q hasta un estado de aceptación.

$$E(q) = \{ \omega \in \Sigma^* \mid q \xRightarrow{\omega} q'; q' \in F \}$$

Ejemplo 7: Consideremos el siguiente AEF:

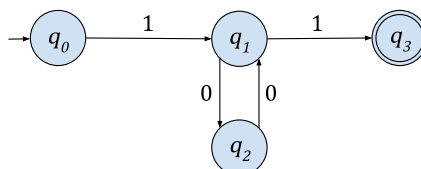


Fig. 29. Diagrama de un AEF y sus conjuntos finales.

El conjunto final $E(q_0)$ es el conjunto de strings que llevan a M desde el estado q_0 hasta el estado de aceptación. Un string perteneciente a $E(q_0)$ debe consistir de un "uno", seguido de algún string que conduzca a M desde el estado q_1 al estado q_3 . Podemos escribir entonces que:

$$E(q_0) = 1 \cdot E(q_1) \quad (1)$$

Similarmente, un string que pertenece a $E(q_2)$ deberá ser un "cero" seguido de algún string que pertenezca a $E(q_1)$.

$$E(q_2) = 0 \cdot E(q_1) \quad (2)$$

Finalmente, un string perteneciente a $E(q_1)$ puede ser un simple "uno" seguido directamente del estado de aceptación q_3 , o un "cero" seguido de un string perteneciente a $E(q_2)$.

$$E(q_1) = 0 \cdot E(q_2) \cup 1 \quad (3)$$

Las ecuaciones (1), (2) y (3) constituyen un sistema de *ecuaciones de conjuntos*, que deben ser satisfechas por los conjuntos finales del aceptor M .

Notemos que el conjunto final del estado q_3 es λ , dado que q_3 es un estado de aceptación y no hay transiciones que salgan de q_3 . Luego, la relación (3) podría escribirse en forma alternativa como:

$$\begin{aligned} E(q_1) &= 0 \cdot E(q_2) \cup 1 \cdot E(q_3) \\ E(q_1) &= \lambda \end{aligned}$$

El lenguaje que la máquina M reconoce es fácilmente expresable en términos de conjuntos finales: Consiste de aquellos strings pertenecientes al conjunto final asociado con el estado inicial de M , es decir, $L(M) = E(q_0)$.

Propiedades fundamentales de los conjuntos finales

- El conjunto final correspondiente a un estado de aceptación siempre contiene el string vacío.
- En forma inversa, si un estado q no es un estado de aceptación, luego $\lambda \notin E(q)$, dado que al menos se requiere algún símbolo de entrada para alcanzar otro estado de M .
- Ahora supongamos que el string ω , no vacío, pertenece a $E(q)$; luego, debe haber una secuencia de estados admisibles para ω , consistente de al menos una transición de estado. Si s es la primera letra de ω , luego $\omega = s \cdot \varphi$ y la secuencia de estados tiene la siguiente forma:

$$q \xrightarrow{s} q' \xrightarrow{\varphi} q''$$

$$\text{donde } q'' \in F.$$

Luego, se deduce que hay un estado q' de M , tal que $q \xrightarrow{s} q'$ es una transición de M y que el string φ pertenece al conjunto final de q' : $\varphi \in E(q')$. Esta proposición también se cumple si uno la analiza en forma inversa.

- Finalmente, un string ω es aceptado por M si y sólo si M tiene una secuencia de estados admisibles $q_0 \xrightarrow{\omega} q'$ donde $q' \in F$. Luego, se concluye que el *lenguaje reconocido* por M es el conjunto final correspondientes al estado inicial q_0 de M .

Proposición 1: Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AEF, y sea $E(q')$ el conjunto final correspondiente al estado q' . Luego:

1. $\lambda \in E(q)$ si y sólo si $q \in F$.
2. Si $\omega = s\varphi$, luego $\omega \in E(q)$ si y sólo si $\varphi \in E(q')$ y $q \xrightarrow{s} q'$ es una transición de M para algún $q' \in Q$.
3. $L(M) = E(q_0)$; q_0 es el estado inicial de M .

Proposición 2: Los conjuntos finales de un AEF $M = (Q, \Sigma, \delta, q_0, F)$ satisfacen un sistema de ecuaciones lineales de conjuntos (por derecha). Por cada estado $q \in Q$:

$$E(q) = \bigcup_{q' \in Q} m(q, q') E(q') \cup n(q)$$

donde:

$m(q, q') = s$; tal que M tiene una transición $q \xrightarrow{s} q'$

$$n(q) = \begin{cases} \lambda & \text{si } q \in F \\ \emptyset & \text{en otro caso} \end{cases}$$

Cada conjunto de coeficientes $m(q, q')$ contiene los símbolos de entrada que pueden llevar a la máquina M desde el estado q al q' .

Las ecuaciones lineales de conjuntos son análogas a las ecuaciones lineales algebraicas, donde la analogía es la siguiente:

Ecuaciones lineales de Conjuntos	Ecuaciones Algebraicas lineales
Unión	Adición
Concatenación	Multiplicación

con la única salvedad que la *concatenación no es conmutativa*.

Las ecuaciones son lineales pues los conjuntos finales, que son las incógnitas, aparecen *como máximo* una vez en cada término. A su vez se denominan *lineales por derecha* porque los conjuntos que constituyen las incógnitas se encuentran a la derecha de cada término que compone el lado derecho de la ecuación. El conjunto de ecuaciones obtenidas a partir de un AEF puede ser resuelto para dar expresiones regulares para los conjuntos finales en términos de los conjuntos de coeficientes de las ecuaciones.

Cuando desarrollamos el sistema de ecuaciones correspondiente a un aceptor, es importante notar que el conjunto final correspondiente a un estado trampa de no aceptación es vacío, y por lo tanto aquellos términos que incluyan ese tipo de conjuntos finales pueden ser eliminados del conjunto de ecuaciones, sin afectar su solución.

También, si q es un *estado inaccesible*, luego el conjunto de ecuaciones para $E(q)$ puede ser omitido del sistema de ecuaciones sin afectar la solución para $L(M)$.

Si M es un AEF que posee la transición:

$$q \xrightarrow{s} q'$$

luego, de acuerdo con la Proposición 1, el string $s\varphi$ pertenece al conjunto $E(q)$ si φ pertenece a $E(q')$. Luego, podemos concluir que:

$$E(q) \supseteq s \cdot E(q') \quad (4)$$

Si G es una gramática lineal por derecha que tiene una producción del tipo:

$$A \rightarrow sB$$

luego el string $\omega = s\varphi$ será derivable a partir de A si y sólo si φ es derivable a partir de B , y por lo tanto podemos afirmar que:

$$L(G, A) \supseteq s \cdot L(G, B) \quad (5)$$

La similitud entre las relaciones (4) y (5) sugiere que, relacionando las reglas de una gramática lineal por derecha con las transiciones de un aceptor, estaremos en condiciones de identificar los conjuntos finales de un aceptor con los conjuntos de strings terminales obtenidos a partir de los símbolos no terminales de la gramática.

Ejemplo 8: Consideremos la gramática:

G:	$S \rightarrow A$	(1)	$C \rightarrow 0B$	(4)
	$A \rightarrow 1B$	(2)	$B \rightarrow 1$	(5)
	$B \rightarrow 0C$	(3)		

A partir de la regla S de G nos encontramos que $L(S) = L(A)$, donde escribimos $L(G, X)$ simplemente como $L(X)$. A partir de la regla (2) tenemos:

$$L(A) = 1 \cdot L(B) \quad (i)$$

y la (4)

$$L(C) = 0 \cdot L(B) \quad (ii)$$

Las dos reglas asociadas a B implican que

$$L(B) = 0 \cdot L(C) \cup 1 \quad (iii)$$

Las relaciones (i), (ii) y (iii) constituyen un sistema de ecuaciones lineales por derecha similares a las formuladas para el aceptor del **Ejemplo 7**, con excepción de que aquí las incógnitas son $L(A)$, $L(B)$ y $L(C)$ en lugar de $E(q_0)$, $E(q_1)$ y $E(q_2)$. Dado que la solución de un sistema es también solución del otro, y que $L(M) = E(A)$ y que $L(G) = L(A)$, luego el lenguaje generado por G es exactamente equivalente al lenguaje reconocido por M . La igualdad de los lenguajes definidos por M y G surge de la relación que existe entre la estructura de M y las producciones en G .

En M	En G
$q_0 \xrightarrow{1} q_1$	$A \rightarrow 1B$
$q_1 \xrightarrow{0} q_2$	$B \rightarrow 0C$
$q_2 \xrightarrow{0} q_1$	$C \rightarrow 0B$
$q_1 \xrightarrow{0} q_3, q_3 \in F$	$B \rightarrow 1$
q_0 estado inicial de M	$S \rightarrow A$

Debido a esta relación, *existe una correspondencia uno a uno* entre las secuencias de estados admisibles de M y las derivaciones en G . Para cada $k \geq 0$, la derivación

$$S \Rightarrow A \Rightarrow 1B \Rightarrow 10C \Rightarrow 100B \Rightarrow \dots \Rightarrow 1(00)^k B \Rightarrow 1(00)^k 1$$

corresponde a la secuencia de estados de aceptación:

$$q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_2 \xrightarrow{0} q_1 \xrightarrow{0} \dots \xrightarrow{0} q_1 \xrightarrow{1} q_3$$

para el string $1(00)^k 1$.

El ejemplo anterior ilustra el principio empleado para construir una gramática lineal por derecha para el lenguaje reconocido por un aceptor de estados finitos arbitrario M .

La gramática posee una letra no terminal asociada con cada estado del aceptor:

$$N = \{N_q \mid q \in Q\}$$

Las producciones de la gramática se obtienen de las transiciones de M de forma tal que:

$$N_q = \omega \text{ sí y solo si } \omega \in E(q), \omega \neq \lambda$$

Las reglas de construcción se enuncian en la siguiente tabla y se explicarán en el **Ejemplo 9**.

Construcción de una gramática lineal por derecha a partir de un AEF

Dado: Un AEF $M = (Q, \Sigma, \delta, q_0, F)$ con conjuntos finales $\{E(q) \mid q \in Q\}$

Construir: Una gramática lineal $G = (N, T, P, S)$ con $N = \{N_q \mid q \in Q\}$ tal que $L(G, N_q) = E(q)$, $q \in Q$

Tabla 1: Reglas de Construcción de una gramática lineal por derecha a partir de un AEF

REGLA	SI M TIENE	LUEGO G TIENE	RAZÓN
1	$\{q_0\} \cap F \neq \emptyset$	$S \rightarrow \lambda$	$\lambda \in L(M)$
2	q_0 estado inicial de M	$S \rightarrow N_{q_0}$	$E(q) \subseteq L(M)$
3	$q \xrightarrow{s} q'; q' \in F$	$N_q \rightarrow s$	$s \in E(q)$
4	$q \xrightarrow{s} q'$	$N_q \rightarrow sN_{q'}$	$E(q) \supseteq sE(q')$

De acuerdo con la definición dada oportunamente, una gramática G construida a partir de la Tabla 1 no es estrictamente lineal por derecha debido a la existencia de producciones del tipo $S \rightarrow N_q$ que resultan de la aplicación de la regla 2.

Recordatorio: Una gramática lineal por derecha contiene solamente producciones que son lineales por derecha. Una producción lineal por derecha cumple una de las formas siguientes:

$$\begin{aligned} A &\rightarrow Ba \\ A &\rightarrow a \end{aligned}$$

donde:

$$\begin{aligned} A &\in (N \cup \{S\}) \\ B &\in N \\ a &\in T \end{aligned}$$

Sin embargo, si a G le adicionamos la producción $S \rightarrow \omega$ por cada producción $N_q \rightarrow \omega$ (que resultan de aplicar las reglas 3 y 4 de la Tabla 1), podemos eliminar la producción $S \rightarrow N_q$ sin afectar el lenguaje generado por la gramática. Así, cualquier gramática generada de acuerdo con la Tabla 1 puede ser transformada, si así se lo desea, en una gramática que es estrictamente lineal por derecha y que genera el mismo lenguaje.

Ejemplo 9: Debemos construir una gramática lineal por derecha G , a partir del AEF que se muestra en la siguiente figura:

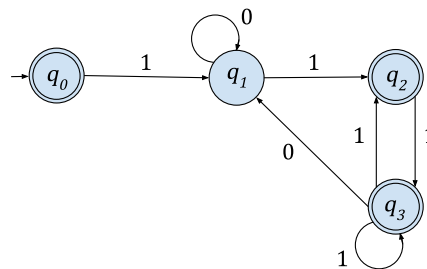


Fig. 30. Diagrama de un AEF para construcción de la gramática asociada.

Vamos a indicar los conjuntos finales $E(q_0)$, $E(q_1)$, $E(q_2)$, $E(q_3)$, de este aceptor, simplemente con q_0 , q_1 , q_2 , q_3 respectivamente. Estos conjuntos están relacionados mediante las ecuaciones de conjuntos lineales por derecha:

$$\begin{aligned} q_0 &= 1q_1 \cup \lambda \\ q_1 &= 0q_1 \cup 1q_2 \\ q_2 &= 1q_3 \cup \lambda \\ q_3 &= 1q_3 \cup 1q_2 \cup 0q_1 \cup \lambda \end{aligned}$$

$$y L(M) = q_0$$

Los símbolos no terminales de G son $N = \{N_{q_0}, N_{q_1}, N_{q_2}, N_{q_3}\}$ en correspondencia con los conjuntos finales de M . Las producciones de G se obtienen empleando las reglas de la Tabla 1.

1. Dado que q_0 es un estado final, G deberá tener la producción $S \rightarrow \lambda$, tal como lo indica la regla de construcción 1.
2. $L(M)$ se corresponde con el conjunto final del estado inicial q_0 . Debido a la regla de construcción 2, G tiene la siguiente producción: $S \rightarrow N_{q_0}$
3. De acuerdo con la regla de construcción 3, G tendrá producciones del tipo $N_q \rightarrow s$, toda vez que $s \in E(q)$, o como se lo indica en la Tabla 1: $s \in E(q)$ si y solo si M tiene una transición $q \xrightarrow{s} q'$ con $q' \in F$.

En el caso del estado q_1 , M tiene $q_1 \xrightarrow{1} q_2$, con $q_2 \in F$ y por lo tanto $1 \in E(q_1)$. Consecuentemente, G tiene la producción $N_{q_1} \rightarrow 1$.

Similarmente, las transiciones:

$$q_2 \xrightarrow{1} q_3, \quad q_3 \xrightarrow{1} q_3, \quad q_3 \xrightarrow{1} q_2$$

donde q_2 y q_3 son estados finales, requieren que G tenga las producciones:

$$\begin{aligned} N_{q_2} &\rightarrow 1 \\ N_{q_3} &\rightarrow 1 \end{aligned}$$

4. Consideremos ahora la ecuación de conjuntos:

$$q_1 = 0q_1 \cup 1q_2$$

Un string no vacío ω pertenece a $E(q_1)$, si y sólo si se cumple que $\omega = 0\varphi$ ó $\omega = 1\varphi$ y que φ pertenezca a $E(q_1)$ ó $E(q_2)$ respectivamente. En G el requerimiento correspondiente se alcanza incluyendo las producciones:

$$\begin{aligned} N_{q_1} &\rightarrow 0N_{q_1} \\ N_{q_1} &\rightarrow 1N_{q_2} \end{aligned}$$

de acuerdo con la regla de construcción 4. Las demás ecuaciones de conjuntos requieren las producciones:

$$\begin{aligned} N_{q_0} &\rightarrow 1N_{q_1} \\ N_{q_2} &\rightarrow 1N_{q_3} \\ N_{q_3} &\rightarrow 1N_{q_3} \\ N_{q_3} &\rightarrow 1N_{q_2} \\ N_{q_3} &\rightarrow 0N_{q_1} \end{aligned}$$

Finalmente, la gramática G se resume por las siguientes reglas de la izquierda. Renombrando los No Terminales obtenemos la gramática de la derecha: $N_{q_0} \rightsquigarrow A, N_{q_1} \rightsquigarrow B, N_{q_2} \rightsquigarrow C, N_{q_3} \rightsquigarrow D$:

G : Regla 1: $S \rightarrow \lambda$ Regla 2: $S \rightarrow N_{q_0}$ Regla 3: $N_{q_1} \rightarrow 1$ $N_{q_2} \rightarrow 1$	G : Regla 1: $S \rightarrow \lambda$ Regla 2: $S \rightarrow A$ Regla 3: $B \rightarrow 1$ $C \rightarrow 1$
--	--

$N_{q_3} \rightarrow 1$ Regla 4: $N_{q_0} \rightarrow 1N_{q_1}$ $N_{q_1} \rightarrow 0N_{q_1}$ $N_{q_1} \rightarrow 1N_{q_2}$ $N_{q_2} \rightarrow 1N_{q_3}$ $N_{q_3} \rightarrow 1N_{q_3}$ $N_{q_3} \rightarrow 1N_{q_2}$ $N_{q_3} \rightarrow 0N_{q_1}$	$D \rightarrow 1$ Regla 4: $A \rightarrow 1B$ $B \rightarrow 0B$ $B \rightarrow 1C$ $C \rightarrow 1D$ $D \rightarrow 1D$ $D \rightarrow 1C$ $D \rightarrow 0B$
--	--

La relación entre las secuencias de estados en M y las derivaciones en G se ilustran en la siguiente figura, donde se muestra la secuencia de estados de aceptación para el string 1001:

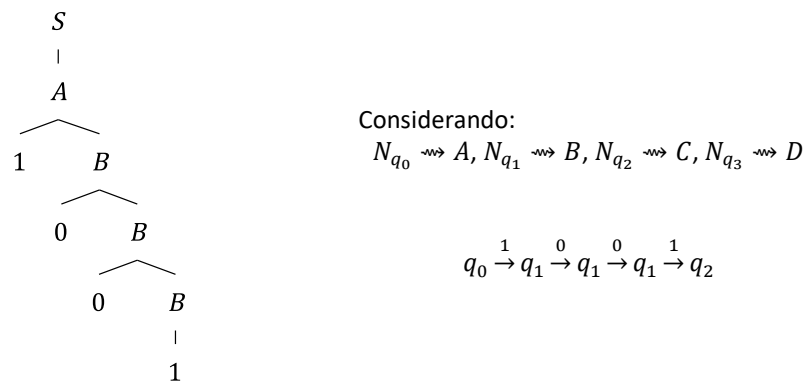


Fig. 31. Relación entre las secuencias de estados en M y las derivaciones en G .

Puede observarse que los símbolos no terminales de G aparecen en las derivaciones en el mismo orden que los estados correspondientes aparecen en las secuencias de estados.

El ejemplo anterior (**Ejemplo 9**) ilustra la correspondencia uno a uno que existe entre las secuencias de estado de un aceptor y las derivaciones en la gramática. Luego, todo esto puede demostrarse formalmente.

Teorema 2: Por cada AEF M se puede construir una gramática lineal por derecha G tal que $L(G) = L(M)$.

Corolario 2.1: Las secuencias de estados en M desde q_0 a $q' \in F$ están en una correspondencia uno a uno con las derivaciones de strings terminales a partir de S en G .

Corolario 2.2: Por cada q , el conjunto final $E(q)$ de M y el símbolo no terminal N_q de G , satisfacen la relación $L(G, N_q) = E(q)$.

Las reglas de construcción dadas en la Tabla anterior pueden revertirse para dar lugar a un aceptor que reconoce el lenguaje generado por cualquier gramática lineal por derecha. Las reglas se dan a conocer en la Tabla 2 y se explican en el **Ejemplo 10**. Por simplicidad, hemos asumido que cada regla que involucra a S en G tiene la forma $S \rightarrow \lambda$ ó $S \rightarrow A$.

Dada: Una gramática lineal $G = (N, T, P, S)$

Construir: Un AEF $M = (Q, T, \delta, q_0, \{q_f\})$ con $Q = \{q_A / A \in N\}$, con conjuntos finales $\{E(q) / q \in Q\}$ tal que $L(G, A) = E(q_A), A \in N \cup \{S\}$

Tabla 2: Reglas de Construcción de un AEF a partir de una gramática lineal por derecha.

Regla	Si G tiene	luego M tiene	Razón
1	$A \rightarrow sB$	$q_A \xrightarrow{s} q_B$	$L(G, A) \supseteq sL(G, B)$
2	$A \rightarrow s$	$q_A \xrightarrow{s} q_f$	$s \in L(G, A)$
3	$S \rightarrow A$	q_A es inicial	$L(G, A) \subseteq L(G)$
4	$S \rightarrow \lambda$	q_f es inicial	$\lambda \in L(G)$

La regla 4 comete una falta con la definición formal de un AEF, permite tener dos estados iniciales en vez de solo uno como lo especifica la definición. Sin embargo, podemos considerar una falta no tan grave desde el momento en que esta situación puede reemplazarse agregando una transición λ desde el estado q_A al estado q_f .

Ejemplo 10: Consideremos el lenguaje $L = a^*b^*$

Emplearemos la letra no terminal A para denotar strings en a^*b^* de acuerdo con la siguiente gramática:

$$\begin{array}{ll}
 G: S \rightarrow \lambda & A \rightarrow aA \\
 & S \rightarrow A & A \rightarrow aB \\
 & & A \rightarrow bB \\
 & & B \rightarrow bB \\
 & & A \rightarrow a \\
 & & A \rightarrow b \\
 & & B \rightarrow b
 \end{array}$$

Queremos construir el AEF que reconoce el lenguaje generado por esta gramática.

- El aceptor M tendrá los estados q_A, q_B y q_f .
- Las transiciones se disponen de forma tal que los strings denotados por cada letra no terminal perteneciente a G , constituyan los conjuntos finales del estado correspondiente en M .

A partir de las producciones $A \rightarrow aA$, $A \rightarrow aB$ y $A \rightarrow bB$ vemos que el conjunto final asociado a q_A debe inducir los conjuntos aq_A , aq_B y bq_B . Esto se logra incluyendo las siguientes transiciones en M (de acuerdo con la regla 1 de la Tabla 2):

$$q_A \xrightarrow{a} q_A, \quad q_A \xrightarrow{a} q_B, \quad q_A \xrightarrow{b} q_B$$

De acuerdo con la misma regla incluimos la siguiente transición:

$$q_B \xrightarrow{b} q_B.$$

- Luego, las producciones $A \rightarrow a$ y $A \rightarrow b$ requieren que los símbolos a y b se encuentren en el conjunto final correspondiente a q_A . Similarmente la producción $B \rightarrow b$ requieren que el símbolo b se encuentre en el conjunto final correspondiente a q_B . Eso se logra incluyendo las transiciones:

$$q_A \xrightarrow{a} q_f, \quad q_A \xrightarrow{b} q_f, \quad q_B \xrightarrow{b} q_f$$

en la máquina A , donde q_f es el estado de aceptación, de acuerdo con lo que especifica la regla 2 de la Tabla 2.

- La producción $S \rightarrow A$ requiere que $L(M)$ contenga al conjunto final asociado a A . Luego q_A será el estado inicial de M , como lo requiere la regla 3 de la Tabla 2.
- Finalmente, la producción $S \rightarrow \lambda$ requiere que q_f sea un estado inicial, de acuerdo a lo que especifica la regla 4 de la tabla antes mencionada.

Luego, el aceptor buscado tiene el siguiente diagrama de estados:

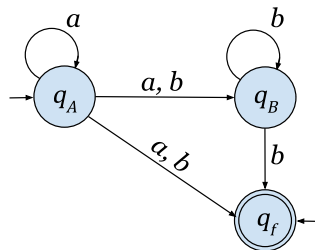


Fig. 32. Diagrama de estados equivalente a la gramática del ejemplo 10.

Luego, este ejemplo ilustra naturalmente cómo una gramática se corresponde con el diagrama de estados de un aceptor no determinístico, y así por qué hemos permitido un comportamiento no determinístico para estos aceptores. Una vez construido éste, es siempre posible construir un aceptor determinístico equivalente, y este procedimiento siempre será más simple que hacerlo directamente a partir de la gramática.

Teorema 3: Por cada gramática lineal por derecha G , uno puede construir un AEF M , tal que $L(M) = L(G)$.

Los *Teoremas 2 y 3* permiten asentar la equivalencia, en términos de la clase de lenguajes que ellos definen, entre los AEF y las gramáticas lineales por derecha. En puntos anteriores dijimos que las gramáticas regulares incluyen también a las gramáticas lineales por izquierda.

Lo que resta es entonces probar que las gramáticas lineales por izquierda son equivalentes a las gramáticas lineales por derecha y así que los AEF son equivalentes a las gramáticas regulares. Esto se hace en forma muy simple.

Equivalencia de gramáticas Regulares lineales por izquierda y derecha

Supongamos que G es una gramática lineal por derecha. Sea G' una gramática lineal por izquierda especificada por la siguiente construcción:

Tabla 3: Reglas de conversión de una gramática lineal por derecha a una gramática lineal por izquierda.

Si G tiene	luego G' tiene
$S \rightarrow sA$	$A \rightarrow s$
$A \rightarrow sB$	$B \rightarrow As$
$A \rightarrow s$	$S \rightarrow As$
$S \rightarrow s$	$S \rightarrow s$
$S \rightarrow \lambda$	$S \rightarrow \lambda$

Esta correspondencia uno a uno entre las producciones de G y las de G' establece una correspondencia uno a uno entre derivaciones en G y en G' .

Una derivación en G :

$$S \Rightarrow s_1 A_1 \Rightarrow s_1 s_2 A_2 \Rightarrow \cdots \Rightarrow s_1 s_2 \dots s_k A_k \Rightarrow s_1 s_2 \dots s_k s_{k+1}$$

se corresponde a la siguiente derivación en G' :

$$S \Rightarrow A_k S_{k+1} \Rightarrow A_{k-1} S_k S_{k+1} \Rightarrow \dots \Rightarrow A_1 S_S \dots S_k S_{k+1} \Rightarrow S_1 S_2 \dots S_k S_{k+1}$$

luego $L(G) = L(G')$.

Ejemplo 11: Consideremos la siguiente gramática lineal por derecha:

$G: S \rightarrow \lambda$	(1)	$A \rightarrow aA$	(7)
$S \rightarrow aA$	(2)	$A \rightarrow aB$	(8)
$S \rightarrow aB$	(3)	$B \rightarrow aB$	(9)
$S \rightarrow bB$	(4)	$A \rightarrow a$	(10)
$S \rightarrow a$	(5)	$B \rightarrow b$	(11)
$S \rightarrow b$	(6)		

para la cual $L(G) = a^*b^*$. Siguiendo las normas ya dadas, una gramática lineal por izquierda equivalente a G es:

$G': S \rightarrow \lambda$	(1')	$A \rightarrow Aa$	(7')
$A \rightarrow a$	(2')	$B \rightarrow Aa$	(8')
$B \rightarrow a$	(3')	$B \rightarrow Ba$	(9')
$B \rightarrow b$	(4')	$S \rightarrow Aa$	(10')
$S \rightarrow a$	(5')	$S \rightarrow Bb$	(11')
$S \rightarrow a$	(6')		

Las derivaciones de la sentencia aabbb de acuerdo a G y G' son:

$$\begin{aligned} G: S &\Rightarrow aA \Rightarrow aaB \Rightarrow aabB \Rightarrow aabbB \Rightarrow aabbb \\ G': S &\Rightarrow Bb \Rightarrow Bbb \Rightarrow Bbbb \Rightarrow Aabbb \Rightarrow aabbb \end{aligned}$$

Las gramáticas lineales por izquierda también pueden ser obtenidas directamente a partir de los aceptores de estados finitos mediante una construcción similar a las gramáticas lineales por derecha.

Para ello uno define el *conjunto inicial* $B(q)$, asociado al estado q como el conjunto de strings que hacen evolucionar al aceptor hasta el estado q , a partir de un estado inicial.

$$B(q) = \{\omega \mid q_0 \xrightarrow{\omega} q, \quad q_0 \text{ es el estado inicial de } M\}$$

La gramática lineal por izquierda G tiene símbolos no terminales $N = \{N_q \mid q \in Q\}$ y producciones elegidas de forma tal que $L(G, N_q) = B(q)$. Las reglas de construcción, las ecuaciones de conjuntos y otros aspectos de este enfoque son análogos a los ya desarrollados.

Los principales resultados obtenidos hasta ahora se resumen por medio de las siguientes afirmaciones:

1. El lenguaje L es reconocido por algún aceptor de estados finitos.
2. El lenguaje L es generado por alguna gramática lineal por derecha.
3. El lenguaje L es generado por alguna gramática lineal por izquierda.

Además, podemos agregar las siguientes afirmaciones correspondientes a gramáticas regulares:

1. A partir de una gramática lineal por derecha, uno siempre puede construir una gramática lineal por izquierda que genere el mismo lenguaje y viceversa.
2. Siempre se puede determinar cuándo dos gramáticas regulares generan el mismo lenguaje: Generan el mismo lenguaje sí y sólo sí sus aceptores determinísticos correspondientes son equivalentes.
3. Siempre se puede determinar cuándo una gramática regular G genera algún tipo de string, es decir averiguar si $L(G)$ es vacío o no. $L(G)$ es vacío sí y sólo sí no existe ningún camino entre el estado inicial y el estado final del diagrama de estados del aceptor correspondiente.

Expresiones Regulares y Aceptores de Estados Finitos

El diagrama de estados de un AEF y las producciones de una gramática regular proveen solamente descripciones indirectas de la estructura de los lenguajes regulares. Queremos poder expresar un lenguaje regular explícitamente en términos de conjuntos de strings simples. A continuación, veremos que esto es siempre posible.

Por cierto, las operaciones de conjuntos unión, concatenación y clausura son suficientes para expresar cualquier lenguaje regular en términos de "singletons" de símbolos alfabéticos (conjunto de símbolos alfabéticos de un único elemento).

Expresiones Regulares

Anteriormente se ha mostrado en forma informal cómo las operaciones de conjuntos unión, concatenación y clausura pueden ser empleadas para formar expresiones con conjuntos de strings.

Por ejemplo, podemos describir un lenguaje L mediante la expresión $(0 \cup 1)^*11$; es decir L es la colección de strings en $\{0,1\}^*$ que finaliza en 11.

Debemos distinguir entre el string de símbolos que constituyen la expresión y el conjunto de strings que la expresión describe. El primero se conoce como **expresión regular** y el segundo como **conjunto regular**. Ya vimos informalmente que dos expresiones regulares distintas pueden describir el mismo conjunto regular.

Por ejemplo, las dos expresiones:

$$1(01)^* \text{ y } (10)^*1$$

describen un conjunto donde todos sus miembros son strings que contienen unos y ceros alternados y que comienzan y finalizan con uno.

Definición: Sea Σ un alfabeto finito. Una *expresión regular* en Σ es cualquier string finito compuesto por símbolos pertenecientes al conjunto:

$$\{a/a \in \Sigma\} \cup \{\cup, *, (,), \lambda, \emptyset\}$$

y que puede ser formado de acuerdo con las siguientes reglas:

1. λ es una expresión regular.
2. $\emptyset$ es una expresión regular.
3. Si $a \in \Sigma$ entonces a es una expresión regular.
4. Si α y β son expresiones regulares entonces:
 - $(\alpha\beta)$ es una expresión regular
 - $(\alpha \cup \beta)$ es una expresión regular
 - (α^*) es una expresión regular

Una expresión regular describe a un conjunto de acuerdo con nuestra interpretación habitual de operaciones de conjuntos. Como ya se mencionó, los paréntesis se omitirán si es que esto no causa confusión.

Dos expresiones regulares son equivalentes sí y sólo si ellas describen el mismo conjunto de strings. Debemos familiarizarnos con algunas equivalencias simples de expresiones regulares. Además de las más usuales correspondientes a unión y concatenación, las equivalencias más importantes se refieren a la operación de clausura (Estrella de Kleene). Luego, aquí tenemos algunas equivalencias simples de expresiones regulares. α , β y γ son expresiones regulares arbitrarias.

- $(\alpha^*)^* = \alpha^*$
- $\alpha^*\alpha^* = \alpha^*$
- $(\lambda \cup \alpha)^* = \alpha^*$
- $\alpha\alpha^* = \alpha^*\alpha$
- $\alpha\alpha^* \cup \lambda = \alpha^*$
- $\alpha(\beta \cup \gamma) = \alpha\beta \cup \alpha\gamma$
- $\alpha(\beta\alpha)^* = (\alpha\beta)^*\alpha$
- $(\alpha \cup \beta)^* = (\alpha^* \cup \beta^*)^*$
- $(\alpha \cup \beta)^* = (\alpha^*\beta^*)^*$
- $(\alpha \cup \beta)^* = \alpha^*(\beta\alpha^*)^*$
- $\alpha = \beta\alpha \cup \gamma \Leftrightarrow \alpha = \beta^*\gamma$

La validez de cada identidad se deriva de las propiedades de unión, concatenación y clausura.
 En general, la ley distributiva no se verifica para la operación de clausura. Por ejemplo, la relación:

$$(\alpha \cup \beta)^* \neq \alpha^* \cup \beta^*$$

dado que el lado derecho (en oposición al izquierdo) no denota ningún string en el cual tanto α como β aparezcan.

Dada una expresión γ que describa un conjunto X , es posible construir en forma fácil una expresión regular γ^R que describa al conjunto X^R que contiene el reverso de cada string en X .

1. Si γ es $\lambda, \emptyset, a \mid a \in \Sigma$, luego γ^R es γ
2. Si γ es $(\alpha\beta)$, luego γ^R es $(\beta\alpha)$
3. Si γ es $(\alpha \cup \beta)$, luego γ^R es $(\alpha \cup \beta)$
4. Si γ es α^* , luego γ^R es α^* .

Luego, para construir una expresión correspondiente al reverso de un conjunto regular, es solamente necesario revertir el orden de las subexpresiones unidas por la operación de concatenación.

Por ejemplo, si α es:

$$01(0^*11 \cup 1^*00)01 = A$$

luego α^R será:

$$10(110^* \cup 001^*)10 = A^R$$

Así, el reverso de cada conjunto regular es también un conjunto regular.

Reducción y Solución de Sistemas de Ecuaciones de Conjuntos

Por cada aceptor de estados finitos $M = (Q, T, \delta, q_0, F)$ podemos construir un sistema de ecuaciones lineales por derecha de acuerdo con la *Proposición 2*, ya vista, que decía que los conjuntos finales de un AEF satisfacen un sistema de ecuaciones lineales de conjuntos. Si M tiene n estados $q_1, q_2, \dots, q_n$ las ecuaciones de conjuntos:

$$E_k = \bigcup_{j=1}^n m_{kj} E_j \cup n_k$$

con:

$$k = 1..n;$$

E_k es el conjunto final correspondiente al estado q_k .

y además:

$$m_{kj} = \{s \in \Sigma \mid M \text{ tiene una transición } q_k \xrightarrow{s} q_j\};$$

$$n_k = \begin{cases} \lambda & \text{si } q_k \in F \\ \emptyset & \text{en otro caso} \end{cases};$$

Para algunas elecciones de j y k , m_{kj} puede ser vacío, en cuyo caso el término $m_{kj} E_j$ se anula.

A partir de la *Proposición 2*, sabemos que los conjuntos finales de M son solución de un sistema lineal por derecha. Debemos establecer, sin embargo, que esos conjuntos son la única solución.

Luego, nos ocuparemos de la construcción de expresiones regulares para estas soluciones y de la prueba de su unicidad. Si bien nos concentraremos en ecuaciones de conjuntos lineales por derecha, los resultados se extienden fácilmente a los sistemas lineales por izquierda.

La solución de estos sistemas de ecuaciones se asemeja al procedimiento de reducción empleado con ecuaciones algebraicas lineales. Manipularemos una ecuación para poder obtener un conjunto incógnita (por ejemplo E_1)

en términos de las incógnitas restantes. Luego, reemplazaremos cada aparición de E_1 en las ecuaciones restantes, y por lo tanto obtendremos un sistema que tiene una ecuación y una incógnita menos.

Esta etapa de reducción se repite hasta que obtengamos una expresión para la última incógnita (por ej. E_n) en términos de los conjuntos de constantes $\{m_{ij}\}$ y $\{m_k\}$ del sistema de ecuaciones.

Para ser más específicos, mostremos una etapa del procedimiento de reducción del sistema de ecuaciones lineales por derecha. Vamos a elegir E_1 , por lo tanto, vamos a reescribir el sistema con la ecuación correspondiente a E_1 escrita en forma separada.

$$E_1 = m_{11} E_1 \cup \left(\bigcup_{j=2}^n m_{1j} E_j \right) \cup n_1$$

$$E_k = m_{k1} E_1 \cup \left(\bigcup_{j=2}^n m_{kj} E_j \right) \cup n_k$$

con:

$$k = 1..n;$$

Primeramente, vamos a encontrar la solución de la ecuación correspondiente a E_1 . La solución no es obvia, dado que E_1 aparece en ambos lados de la ecuación. De acuerdo con la ecuación asociada a E_1 , un string ω pertenece al conjunto E_1 sí y sólo sí:

$$\omega \in \left(\bigcup_{j=2}^n m_{1j} E_j \right) \cup n_1$$

ó

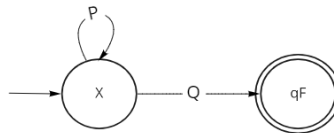
$$\omega = s\varphi \text{ donde } \varphi \in E_1 \text{ y } s \in m_{11}$$

En estas condiciones, podemos reescribir la ecuación para E_1 de la siguiente manera:

$$E_1 = m_{11} E_1 \cup \left(\bigcup_{j=2}^n m_{1j} E_j \right) \cup n_1$$

$$E_1 = (m_{11})^* \left(\bigcup_{j=2}^n m_{1j} E_j \cup n_1 \right)$$

Que puede representarse en forma abstracta de la siguiente manera:



donde:

$$X = E_1$$

$$P = m_{11}$$

$$Q = \bigcup_{j=2}^n m_{1j} E_j \cup n_1$$

Luego la ecuación tiene la siguiente forma:

$$X = PX \cup Q$$

donde X corresponde al conjunto final desconocido. De la última figura es obvio que también se cumple que:

$$X = P^*Q$$

es también una solución.

Esta última relación se denomina también *Regla de Arden*. Si aplicamos esta regla a la ecuación correspondiente a E_1 , obtendremos:

$$E_1 = (m_{11})^* \left(\bigcup_{j=2}^n m_{1j} E_j \cup n_1 \right)$$

Si sustituimos esta expresión correspondiente a E_1 en cada una de las $n - 1$ ecuaciones restantes, obtendremos:

$$E_k = \bigcup_{j=2}^n (m_{k1}(m_{11})^* m_{1j} \cup m_{kj}) E_j \cup m_{k1}(m_{11})^* n_1 \cup n_k$$

con:

$$k = 2..n;$$

Si denominamos:

$$\begin{aligned} m_{kj}' &= m_{k1}(m_{11})^* m_{1j} \cup m_{kj} \\ n_k' &= m_{k1}(m_{11})^* n_1 \cup n_k \end{aligned}$$

obtendremos $n - 1$ ecuaciones lineales por derecha con $n - 1$ incógnitas:

$$E_k = \bigcup_{j=2}^n m_{kj}' E_j \cup n_k'$$

con:

$$k = 2..n;$$

de las cuales E_1 se ha eliminado completamente.

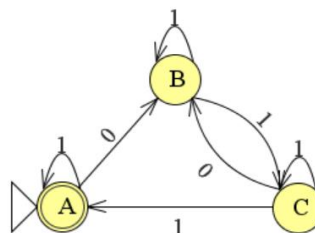
La solución completa del sistema de ecuaciones se encuentra a partir de la aplicación repetida de la etapa de reducción ya descrita.

Indicaremos con E_k el sistema de ecuaciones lineales por derecha, que contiene $n - k$ ecuaciones, obtenidas luego de eliminar las incógnitas $E_1, \dots, E_k$. Cada etapa de reducción transforma el sistema E_k en E_{k+1} mediante la eliminación de E_{k+1} .

Así, el sistema E_{n-1} consiste en una ecuación en una incógnita, el conjunto E_n ; y es resuelto mediante la regla de Arden. La expresión resultante para E_n involucra solamente operaciones de unión, concatenación y clausura que se aplican a conjuntos de constantes (las m 's y las n 's), y es por lo tanto una expresión regular.

Para $k = n - 1, \dots, 1$, sustituimos las expresiones de $E_{k+1}, \dots, E_n$ en la ecuación correspondiente a E_k del sistema E_{k-1} con el fin de obtener una expresión regular para el conjunto incógnita E_k . Procediendo de esta forma, obtenemos la solución completa del sistema de ecuaciones.

Ejemplo 12:



Las ecuaciones lineales por derecha correspondientes a este aceptor son:

- $E(A) = 1 \cdot E(A) \cup 0 \cdot E(B) \cup \lambda$
- $E(B) = 1 \cdot E(B) \cup 1 \cdot E(C)$
- $E(C) = 1 \cdot E(C) \cup 0 \cdot E(B) \cup 1 \cdot E(A)$

Que prescindiendo de la notación de conjunto final obtenemos:

- $A = 1A \cup 0B \cup \lambda$
- $B = 1B \cup 1C$
- $C = 1C \cup 0B \cup 1A$

Nos proponemos hallar una expresión para $L(M) = A$. Por lo tanto, eliminaremos C primero, y luego B . Resolveremos la ecuación correspondiente a C mediante la regla de Arden. El resultado es:

$$C = 1^*(1A \cup 0B)$$

Si sustituimos esta expresión en las ecuaciones que corresponden a A y B , y luego reagrupamos términos, obtendremos un sistema con 2 ecuaciones en 2 incógnitas:

- $A = 1A \cup 0B \cup \lambda$
- $B = (1 \cup 11^*0)B \cup 11^*1A$

Si aplicamos la regla de Arden a la nueva ecuación correspondiente a B encontraremos:

- $B = (1 \cup 11^*0)^*11^*1A$

Empleando esta expresión en la ecuación correspondiente a A y reagrupando términos, llegamos a:

- $A = 1A \cup 0(1 \cup 11^*0)^*11^*1A \cup \lambda$

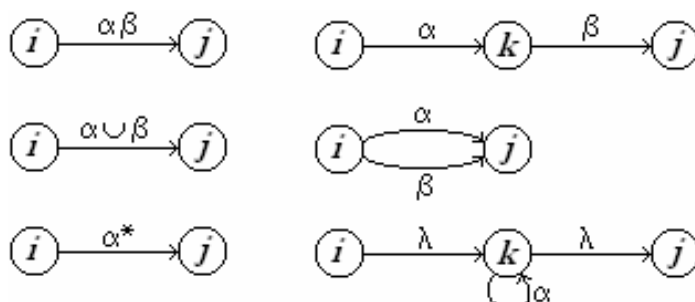
Finalmente, la última aplicación de la regla de Arden nos da lo que deseábamos, o sea una expresión para $L(M) = A$.

$$L(M) = A = (1 \cup 0(1 \cup 11^*0)^*11^*1)^*$$

Durante la aplicación del procedimiento de reducción, los conjuntos incógnita del sistema de ecuaciones, pueden ser eliminados en cualquier orden, y una elección inteligente puede lograr que se ahorren esfuerzos o que se llegue a expresiones más simples. Si bien la forma de las expresiones obtenidas depende del orden seguido durante el proceso de reducción, las expresiones siempre describen el mismo conjunto.

Construcción de un AEF a partir de una ER

Se parte de una máquina M_1 con un único estado inicial, un único estado final, y una transición entre ellos rotulada por la expresión regular en cuestión; el procedimiento consiste en transformar gradualmente M_1 en M_2 , luego en M_3 , etc., hasta llegar a una M_n tal que sus transiciones estén rotuladas por símbolos del alfabeto (o bien por el string vacío).



Ejemplo:

